

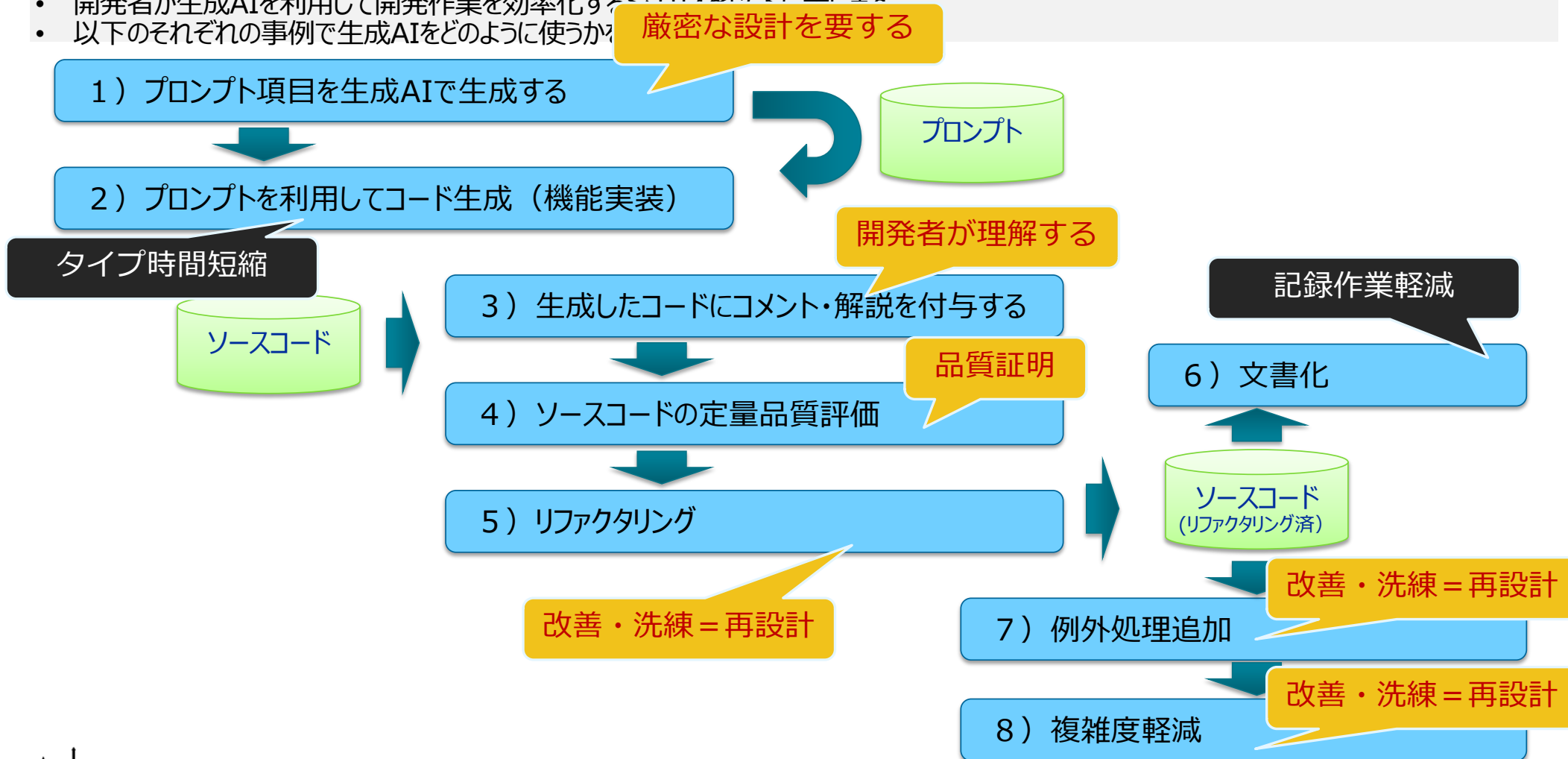
第5回 生成AIとAI駆動開発 #2



利用全体像：開発者が生成AIと協働する – シナリオ全体像



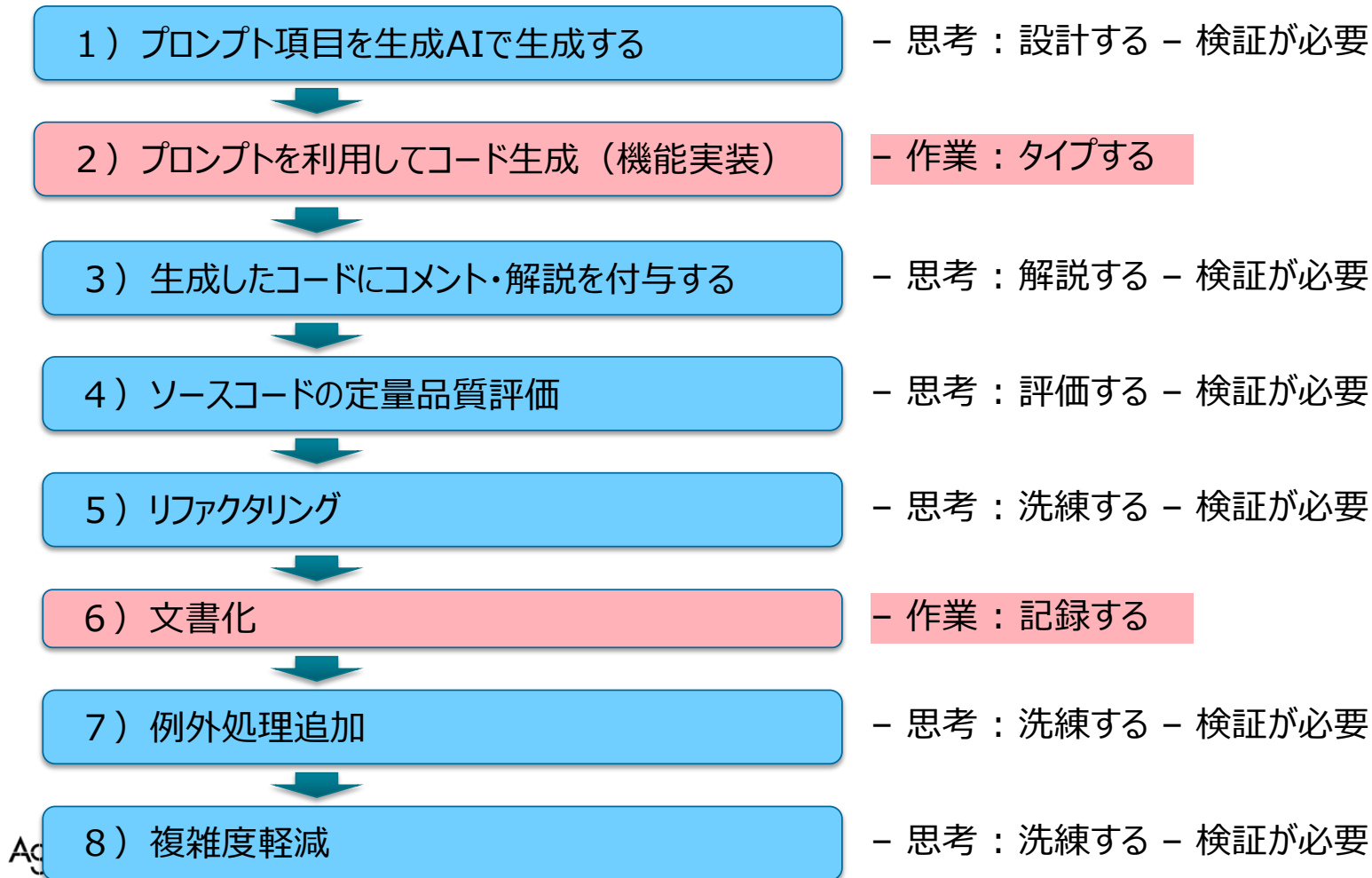
- 開発者が生成AIを利用して開発作業を効率化するシナリオを以下に示します
- 以下のそれぞれの事例で生成AIをどのように使うか



開発者が生成AIと協働する – AI≠生産性向上・時間短縮



- 開発者がAIを使う場合と使わない場合では、効率化できる作業は2) コード生成と、6) 文書化の作業時間の短縮のみであり、トータルではより品質を高めるための「検証」作業が増えるため、時間よりも品質改善活動が増加する傾向にある。



- AIは生産性を高める道具ではない。
- **短縮できる時間は極めて僅か。**
(むしろ増える)
- 品質を高める道具としてAIを利用するべき。

- ① 研究者によると、AIを取り入れると、時間的なプレッシャーやマルチタスクの必要性など、認知的な制約がある場合に仕事量が増加する可能性があるという。
 - にもかかわらず、**組織は労働者の負担を軽減したり、給与にかかる費用を減らしたりできることを期待して、AIの利用を増やしている。**
- ② さらに、AIは事実上常に選択肢の全体的な精度を向上させるが、誤検出など特定のミスを頻発させる可能性もある。
 - 例えば、医療診断の場合、ほとんどの状況において、AIの使用は全体的な精度を向上させ、誤診のリスクを低減させるという論文結果が示されている。
 - しかし、病気がまれで、医療従事者が時間的制約の中で仕事をしている場合、AIを使用すると誤診の発生率が高くなり、AIを使用することで生じる可能性のある「あいまいさ」や不正確さを解決する必要があるため、人間の側により多くの認知的努力が必要になる。

生成AIの意味的欠陥の誤生成の例



- 意味的（セマンティクス）に誤った結果の生成は、生成AIを利用する際には不可避です。これは入力データや参考にする他コードの情報に引きずられることが原因です。

Objective-Cのコードを解析するためのPythonコード

- Pythonコード内の共通ライブラリ読み込み宣言部のコード
- 「コメント生成」を依頼した結果、同じワークスペース内のPythonコードを解析するコードと混同した可能性
- 出現確率が高い「文章」や「字句」を参考にする性質が原因か？

```
21 # 共通ライブラリのディレクトリのモジュールをインポート
22 try:
23     # ライブラリ検索パスとしてMetricsディレクトリを追加
24     sys.path.append(os.path.join(os.path.dirname(__file__), 'metrics'))
25     # Pythonの解析モジュールをインポート
26     from QI_ObjectiveC_Metrics import ObjectiveCMetrics
27
28     # Pythonのキーワードモジュールのインポート
29     sys.path.append(os.path.join(os.path.dirname(__file__), 'keyword_file'))
30     # Pythonのキーワードモジュールのインポート
31     from keyword_file import objc_keywords
32     #import objc_keywords
33
```

- 25行目および28, 30行目でObjective-C解析ライブラリを読み込む箇所のコメントが「Pythonのモジュールををインポート」とコメント生成。（実際はObjective-CのモジュールをImport）
- 当プログラムはPythonで書かれているため「何の言語」を解析するかを混同した？

生成AIの意味的欠陥の誤生成の例：“申し訳ありません”と試行錯誤



- 意味的（セマンティクス）に誤った結果の生成の代表的な例としては、単純なコード変換の際、標準ライブラリが提供されない処理を変換中に「余計な」配慮を加えてしまうことです。

- ファイルの文字コードを検出するメソッド（Python）

```
def detect_encoding(self, file_path: str) -> str:
    """ファイルエンコーディングを検出"""
    try:
        with open(file_path, 'rb') as f:
            raw_data = f.read()
            result = chardet.detect(raw_data)
            encoding = result['encoding']
            confidence = result['confidence']

            # 信頼度が低い場合はUTF-8を試す
            if confidence < 0.7:
                try:
                    raw_data.decode('utf-8')
                    return 'utf-8'
                except UnicodeDecodeError:
                    pass

            return encoding if encoding else 'utf-8'
    except Exception:
        return 'utf-8'
```

開発者：選択したメソッドをC++言語に変換してください。

生成AI：“選択したメソッドをC++言語に変換してください”、17 結果のコードベースを検索しました。

以下の誤生成 + 試行錯誤の形跡の例)

→結果としてバグ残存+大量のコード+解説文章

- 試行錯誤の連続

- 申し訳ありません。既存のファイルを編集する必要があるようです。
- 申し訳ありません。ファイルの最後に改行を追加する必要があります：
- 申し訳ありません。ファイルの修正に問題が発生しています。この問題は技術的な制限によるものですが...
- 申し訳ありません。既存のファイルを編集する必要があるようです。
- すみません、エラーが発生しています。
- すみません、エラーが発生しています。

当該例はScript言語->C++言語変換 = 生成AIが苦手な処理の一つ

- 問題は、左のパイソンコードをC++で書き直す際、PythonをObjective-Cと誤認していることと、標準APIのない処理であるため、実直に処理を書き下している点。

合計140行程度のCPPと20行、README.meが600行程度まで膨れ上がりました。

生成AIの保守性欠陥の誤生成の例：警告残存でも成功報告



- 同時に大量の成果物を生成する処理では、数件の警告が残った状態でも「テスト合格」「生成成功」と報告することがあります。

- ファイルの文字コードを検出するメソッド (Python)

開発者：選択したメソッドをC++言語に変換してください。

生成AI：“選択したメソッドをC++言語に変換してください”、17 結果のコードベースを検索しました。

- 処理概要をここに書く

2. 主要プログラミング言語カテゴリ (6/6)

言語	ステータス	F数	実行時間	特記事項
Python	成功	103	1.67秒	高速処理、完全なCSV出力
Java	成功	1	1.23秒	正常な解析、CSV出力完了
C++	成功	1	即座	12行、0関数、正常処理
C	部分的	1	即座	実行成功、regexエラー警告あり
JavaScript	成功	1	0.003秒	超高速処理、CSV出力正常
TypeScript	成功	1	0.012秒	高速処理、CSV出力正常

- 問題は、左のパイソンコードをC++で書き直す際、PythonをObjective-Cと誤認していることと、標準APIのない処理でため、実直に処理を書き下している点。

合計140行程度のCPPと20行、README.meが600行程度まで膨れ上がりました。

- 生成AIを用いた、いわゆるAI駆動型開発について一般的に言われる利点の多くは、一定の条件が揃わないとメリットを享受できないどころか、生産性と品質を低下させる場合も少なくありません。ここでは、一旦生成AIを利用したソフトウェア開発の「注意点」をまとめて解説します。

生成AIの幻想：

- 1) 開発費用を安く抑えられる。
 - 2) 開発期間を短縮できる。
 - 3) テストを効率化できる可能性がある。
 - 4) 保守運用を効率化することができる。
- ：

– 左記のメリットを達成するには、いくつかの条件が必要です。

1. 開発標準・プロセスが整っていること。
2. ある程度ブラックボックス化を容認すること。
3. 開発者の設計能力が高いこと。（特に仕様化）
4. 法律含めてガードレールが整備されていること。
5. 開発環境が整っていること。
（開発技術の変化に対応できること）

生成AIのメリット・デメリット：幻想と現実「開発費用を安く抑えられる」



- 古くから、「品質を無視すれば全てのソフトウェア開発プロジェクトは成功する」と言われています。開発費用を安く抑えるためのAI利用は

生成AIの幻想：

- 1) 開発費用を安く抑えられる。
 - 2) 開発期間を短縮できる。
 - 3) テストを効率化できる可能性がある。
 - 4) 保守運用を効率化することができる。
- ：

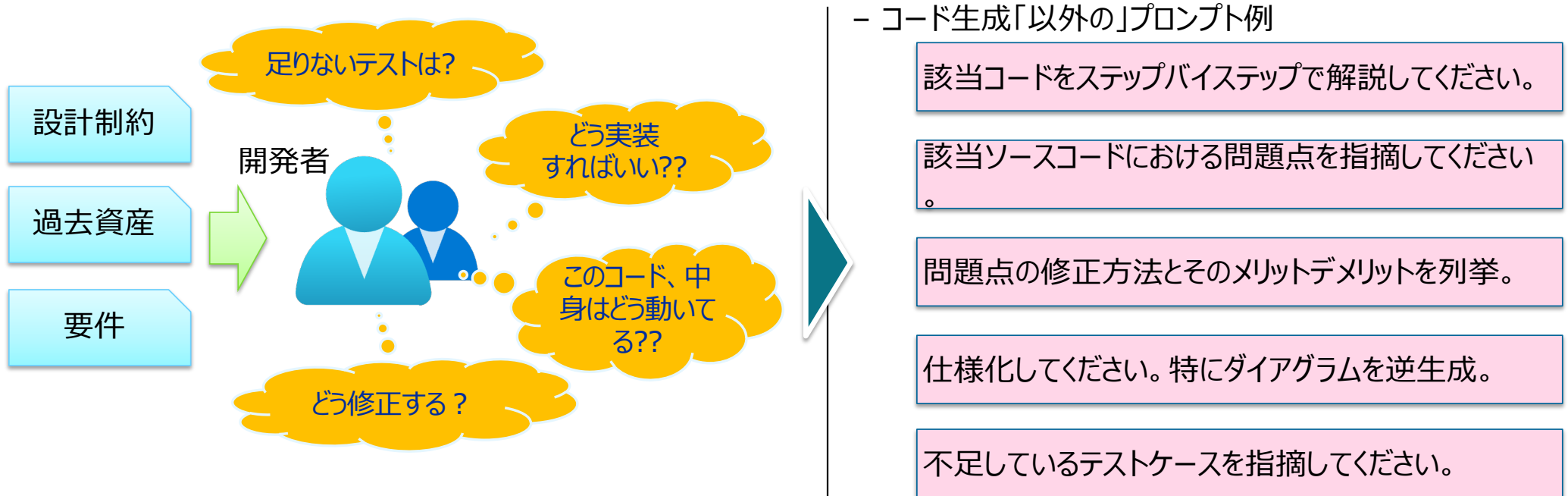
現実には：

- コードが「粗製濫造」されるため、管理対象が従来より増加。
- コード量が増えるため、テストケースが爆発的に増加。
- 大規模ソフトウェアであれば変更影響分析難しい。
- コード実装がブラックボックス化するため問題判別時間が長い。
- 生成されたコードの標準化対応 = 書き直し時間がかかる。
- ドキュメンテーション（設計仕様）との乖離が頻発。
- 同じモデルからの再生成しにくい。同コードの再入手が難しい。

生成AIのメリット・デメリット：では現実に享受できるメリットは？



- 生成AI利用の一番のメリットは、開発者にとっての「AI駆動の知識獲得」です。開発経験の少ない若手のエンジニアにとっては、高速なプロトタイピング、詳細な実装技術獲得、バグ知識、設計技術向上などの「壁打ち相手」「並走者」としてのAI利用が最も利点が多いと言われています。



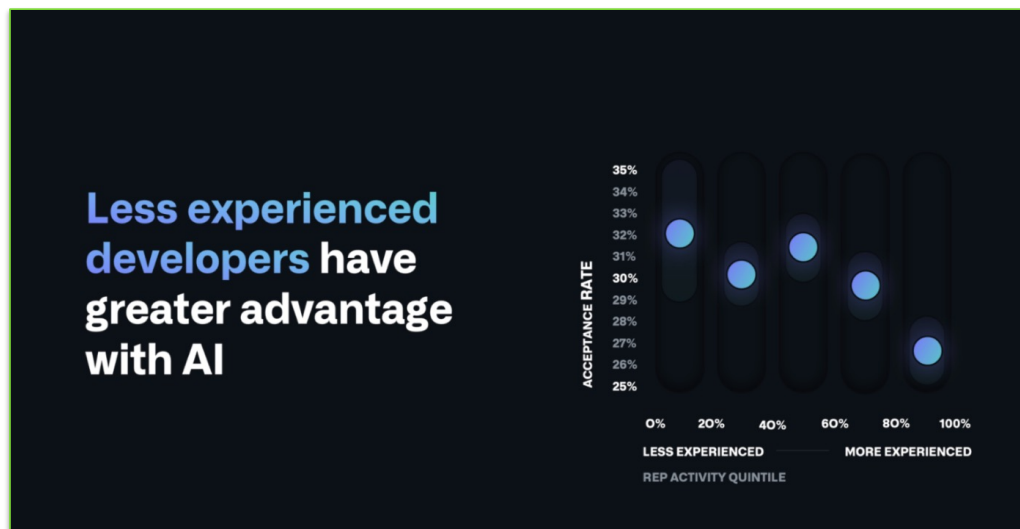
開発者のAI駆動知識獲得：生成AIを「高速プロトタイピングツール」として利用する

- 高速のトライ＆エラーツールとして利用する。
- 実装バリエーション・テストバリエーションの提示は全ての開発者にとってメリット。
- 品質改善＋理解向上系のプロンプトを多用することで開発者の「能力補完」＝**家庭教師**として利用する。

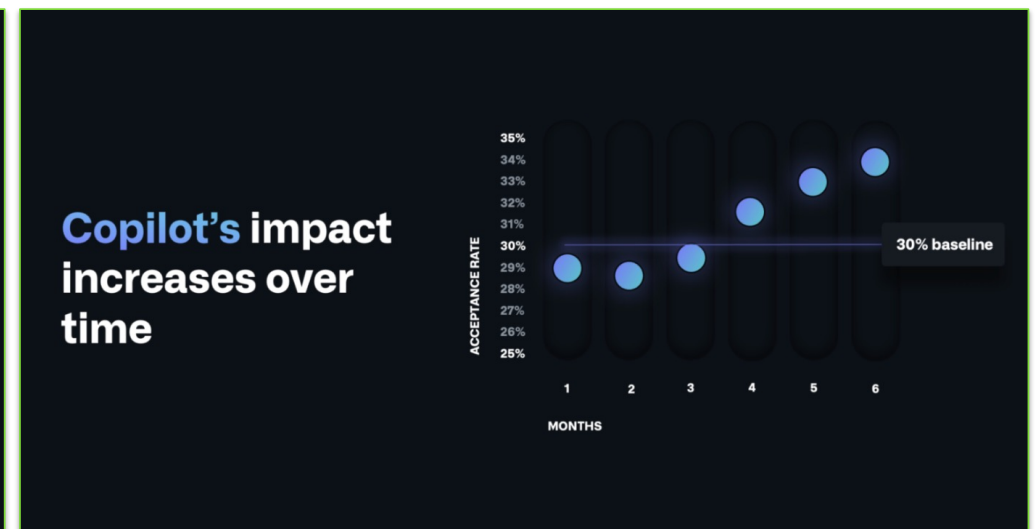
参考) 生成AIのメリット・デメリット : 生成AIの経済性と学習曲線



- GitHub Copilotは開発者の生産性を飛躍的に向上させています。大規模なGitHub Copilotユーザーサンプル (n = 934,533) の分析により、生産性への顕著な影響が明らかになりました。市場投入後1年以内に、ユーザーはGitHub Copilotからのコード提案の約30%を受け入れ、これらの受け入れによって生産性が向上したと報告しています。
- さらに、開発者がツールに慣れるにつれて、時間の経過とともに受け入れ率が着実に上昇することも判明しました。これは、ユーザーがGitHub Copilotを用いたソフトウェア開発に慣れるにつれ、開発者の生産性への影響を継続的に拡大する大きな余地があることを示唆しています。
<https://github.blog/news-insights/research/the-economic-impact-of-the-ai-powered-developer-lifecycle-and-lessons-from-github-copilot/>



**経験の浅い開発者が、経験豊富な開発者よりも相対的に大きな恩恵を受けている。



** GitHub Copilotの提案に対する受諾率の経時変化。

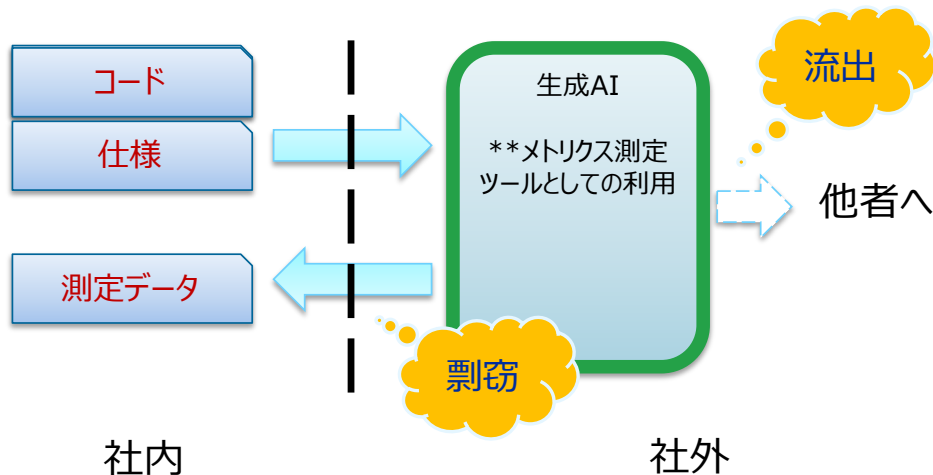
生成AIのメリット・デメリット：生成AIの直接利用と間接利用



- 生成AIへのコードやデータの投入によって知的所有権の流出や剽窃問題に対応するためには、生成AIの直接利用を避け、間接的に利用する方法があります。間接利用とは生成AIを直接生産するのではなく、**目的とする処理に対するプログラムをAIに生成させる「プログラム・ジェネレータ」としての利用**です。

生成AIの直接利用プロセス

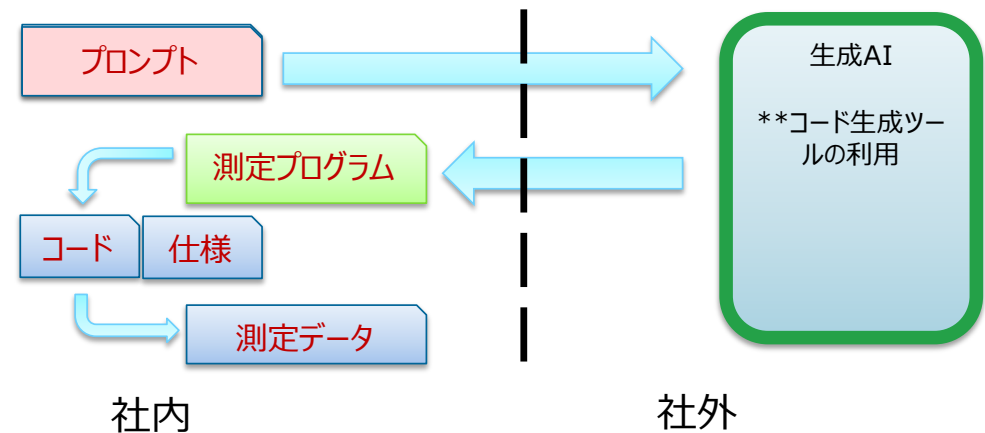
- 入力コードのメトリクスを測定してCSVに保存してください。



- 入力コードや仕様が他所へ流出するリスク
- 測定メトリクスデータの構造や種類・アイデアを他所から剽窃するリスク

間接利用のプロセス=コードジェネレータとして利用

- 入力コードのメトリクスを測定するプログラムを生成してください。
- 生成するプログラムの出力はCSVで出力してください。



- 測定プログラムの生成を依頼
- コードと仕様が生成した測定プログラムの入力にし、測定データを生成（社内）

生成AIの間接利用：生成AIに作業のためのプログラム生成を依頼し、それを実行する

- 知的所有権の流出・剽窃の防止と、再現性・可塑性・保守性が向上するメリットがある。

生成AIのメリット・デメリット：生成結果をチェックする「チェックリスト」生成



- 生成AIを利用する大前提として、AIからの提案を無批判に受け入れることは避け、常に**結果を慎重に吟味する姿勢**が重要です。生成AIを疑う視点を持ちましょう。
- ハルシネーションばかりでなく、それがベストな方法かどうか合わせて確認しましょう。以下は「チェックリスト生成」機能の利用です。

スタイルに関するチェックリストの生成指示プロンプト

生成したコードに対して、コーディングスタイルが適切かどうか確認するためのチェックリストを作成し、チェックリストに従ってチェックしてください。

- ① このプロンプトからは、コーディングスタイルのチェックリスト（マークダウン形式）が生成され、「開発者が利用しやすい」形式のPDFファイルに変換されます。
- ② さらにそのチェックリストによるコーディングスタイルのテストを実施し、その結果をレポートしてくれます。

CODING_STYLE_CHECKLIST.md2025-11-15

①

コーディングスタイルチェックリスト

概要

このチェックリストは、当プロジェクトで生成されたPythonコードのコーディングスタイルが適切かどうかを確認するためのものです。PEP 8 (Python Enhancement Proposal 8) を基準とし、プロジェクト固有の規約も含みます。

1. ファイルレベルのチェック

1.1 ファイルヘッダー

- ☐ ファイルヘッダーコメントが存在する

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

- ☐ ファイル情報ブロックが記載されている
 - Language Type
 - Class Name / File name
 - Author
 - Descriptions
 - Pre-Conditions / Post-Conditions
 - Revision history
- ☐ 作成者と日付が記録されている

該当ファイルの状態:

- ☒ Proc_7_Generate_DataDictionary_from_DD_350NL.py
- ☒ Proc_8_Generate_DataStructureJson_from_DD_350NL.py

1.2 インポート文

- ☐ 標準ライブラリ、サードパーティ、ローカルモジュールの順序でグループ化
- ☐ 各グループは空行で区切られている
- ☐ アルファベット順に並べられている
- ☐ import * を使用していない

推奨される順序:

CODING_STYLE_CHECK_REPORT.md2025-11-15

②

コードスタイルチェック結果レポート

実行日時

2025年11月15日

チェック対象ファイル

- Proc_7_Generate_DataDictionary_from_DD_350NL.py
- Proc_8_Generate_DataStructureJson_from_DD_350NL.py

使用ツール

- Rake8 (最大行数: 100文字)

チェック結果サマリー

検出された問題

エラーコード	件数	説明	重要度
W293	130	空行に空白文字が含まれている	低
E501	7	行が100文字を超過	中
F541	14	f-stringにプレースホルダーがない	低
W291	1	行末に空白文字	低

合計: 162件の指摘

詳細分析

1. W293: 空行に空白文字 (130件)

問題: 空行にスペースやタブが含まれている

影響度: 低

- 機能には影響しない
- エディタによってはファイルが「変更あり」と表示される可能性

修正例:

```
# 修正前
def my_function():
    pass
```

1 / 8

生成結果へのチェックリストの活用：小さな「静的コード解析（SCA）」ツールのジェネレータとして利用する

- 一般的な文法・スタイルなどの品質チェックは自動的にチェックする機能と同時にチェックリストの生成機能を利用する。
- モデル妥当性、設計書-コード間の乖離確認、テストカバレッジ確認など、さまざまなチェックリストが生成できる。

生成AIのメリット・デメリット : CoT(Chain of Thought)プロンプティング



- 生成AIのコード生成能力は、時として問題に対して推論タスクを適用することがあります。これは長く複雑な「〜と仮定して」という仮定の連続で実装ロジックを組み立てる方法です。
- この推論・仮定の連続をプログラム中のコメントとして表現させることによって設計過程やバグの診断などの処理を開発者が理解しやすくなります。

Step-by-Stepを含んだプロンプト (CoT Promptingの例)

Javaでクイックソートアルゴリズムを実装してください。
* ステップバイステップで考えて、各ステップの説明をコメントとしてコードに加えてください。

- ① 生成されたコードに対して「ステップ1〜4」の段階的な処理実装に対するコメントが追記される：実装時に「どう考えて」実装したかを推し量る手掛かりになる。
- ② さらにその実装と整合した仕様書（md形式）を生成することで、内部仕様を検証することも可能：
 - この仕様書に対する自動レビュー・仕様リファクタリングなどもAIで検討することもできる。

①

```
Java
/**
 * 配列の指定された部分（lowからhighまで）をク
 * @param arr 整数配列
 * @param low ソートする範囲の開始インデックス
 * @param high ソートする範囲の終了インデックス
 */
private static void quickSort(int[] arr, int low, int high) {
    // ステップ 1: 再帰のベースケースの確認
    // low >= high の場合、部分配列の要素数が1以下であるため、ソートの必要はない
    if (low < high) {

        // ステップ 2: partition（分割）処理の実行
        // partitionメソッドは、配列をピボットに基づいて分割し、
        // ソート後のピボットの最終的な位置を返す
        int partitionIndex = partition(arr, low, high);

        // ステップ 3: ピボットの左側の部分配列を再帰的にソート
        // partitionIndexの要素はすでに正しい位置にあるため、対象外とする
        quickSort(arr, low, partitionIndex - 1);

        // ステップ 4: ピボットの右側の部分配列を再帰的にソート
        quickSort(arr, partitionIndex + 1, high);
    }
}
```

②

1. quickSort(arr, low, high) (再帰的なソート)

これは、クイックソートの中心となるメソッドです。

- **ステップ1(ベースケース):** if (low < high) の条件で、ソート対象の部分配列に2つ以上の要素があるかを確認します。要素数が0または1であれば、すでにソート済みとみなし、再帰を終了します。
- **ステップ2(分割):** partition(arr, low, high) を呼び出し、部分配列をピボットに基づいて2つのグループに分け、ピボットをその最終的なソート位置に配置します。
- **ステップ3&4(統治):** ピボットの左右にある2つの新しい部分配列に対して、自身 (quickSort) を再帰的に呼び出します。これにより、すべての要素がソートされるまで分割が繰り返されます。

複雑な問題に対するCoTプロンプティング : AIがどう考えたかを検証する手掛かり=「ステップバイステップで」

- コメント追加や.mdファイルを生成することで、生成AI側の「思考の過程」を理解しやすくなる。
- [注意点]** プロジェクトの初期段階の学習には最適だが、プロダクションレベルのコードでは「冗長になる」のでコメント整理が必須。

生成AIのメリット・デメリット : CoT(Chain of Thought)プロンプティング



- 生成AIのコード生成能力は、時として問題に対して推論タスクを適用することがあります。これは長く複雑な「～と仮定して」という仮定の連続で実装ロジックを組み立てる方法です。この推論・仮定の連続をプログラム中のコメントとして表現させることによって設計過程やバグの診断などの処理を開発者が理解しやすくなります。

目的	プロンプト例	解説
実行効率の向上	以下のプログラムの非効率な部分はどこですか？ ステップバイステップ で解説してください。	複雑なアルゴリズムやビジネスロジックの設計において、問題を分解し、各ステップで必要な処理を明確するために使われる。
複雑なバグの特定	以下のプログラムのバグを特定して修正してください。 その方法を ステップバイステップ で解説してください。	プログラムの実行フローをステップ倍ステップで追うことで、非直感的なバグやエラーの原因を特定し、反復的にAI修正を促すことができる。
学習ツールとして	以下のプログラムの処理を ステップバイステップ で説明してください。	コードのレビューのために使うことで、レビュー担当者がプログラムの理解を深め、レビュー効率を向上させることができる。
汎用の「魔法の言葉」	- 多角的な視点で考えてください。 - デザイン思考で考えてください。 - 答えに至るまでの、中間推論ステップを明らかにしてください。	答えをいきなり出すのではなく、一つずつ箇条書きで中間の思考を「検証ように」書き出す。

コンセプトのミスリード : もっともらしいが正しくない推論経路が生成され、誤解を招いたり、誤った結論に至る危険性がある。

費用と労力がかかる : 効果的なCoTプロンプトを設計するには、問題領域とモデルの機能を深く理解する必要があり、より複雑で労力がかかる。

弊害

モデルの過剰適合 : プロンプトの推論のスタイルやパターンにモデルが過剰に適合する潜在的なリスクがあり、様々なタスクに対する一般化機能が低下する可能性がある。

評価と検証 : CoTは解釈可能性と正確性を高めることができるが、推論や理解の質的向上を測定することは困難である。

デバッグポイントの列挙



- 全般的なテスト視点を網羅したのち、具体的なテスト条件を明確化して、より詳細なテストを行うことも可能です。

テスト条件を具体化する例

- テストのフレームワークやライブラリなど、具体的な情報を提供する。
- 期待するテストケースの数を指定する。
- エラーケースやコーナーケースを含む具体的な条件を示す。

@プロンプト :

以下のPython関数に対する単体テストを生成してください。

```
def multiply(x, y):  
    return x * y
```

@プロンプト :

定義したmultiply()に以下の条件を満たす単体テストを作成してください。

- **Unittest**を使用してユニットテストを追加してください。
- エッジケースを含む少なくとも**20種類の入力**でメソッドの挙動をテストしてください。
- テストケースには、有効同値、無効な入力、エラーケース、さまざまな条件を考慮してください。
- テストのソート順序は例外系・無効な入力値など、falseになるケースから実行してください。

ディシジョンテーブルの生成と網羅性確認



- AIが生成するテストコードの品質を高めるために、テストケース設計にディシジョン・テーブルを活用できます。
- 仕様条件を先に明確にして、条件に基づいてディシジョンテーブルを生成するようにAIに指示を出します。

テスト条件を具体化する例

- テストのフレームワークやライブラリなど、具体的な情報を提供する。
- 期待するテストケースの数を指定する。
- エラーケースやコーナーケースを含む具体的な条件を示す。

@プロンプト:

以下のディシジョン・テーブルをMarkdownで作ること。

条件:

- 通常会員とVIP会員がいる。
- VIP会員には平常時10%割引が適用される。
- セール時にはそれぞれ10%と20%の割引が適用される。

期待される出力:

- 行: 条件および各割引率
- 列: パターンの番号
- コンテンツ: True/False, X

条件 / 割引率	パターン 1	パターン 2	パターン 3	パターン 4
条件 1: VIP会員である	F	T	F	T
条件 2: セール時である	F	F	T	T
アクション 1: 割引率 0%	X	-	-	-
アクション 2: 割引率 10%	-	X	X	-
アクション 3: 割引率 20%	-	-	-	X
結果 (会員ランク)	通常会員	VIP会員	通常会員	VIP会員
結果 (状況)	平常時	平常時	セール時	セール時
最終割引率	0%	10%	10%	20%

ディシジョンテーブルの生成と網羅性確認



- もう一つの方法は、AIに直接コードを読み込ませ、IF分岐などを元にディシジョンテーブルを作成することです。
- このメリットは、「プログラムに必要なテストケース数の概算」を簡単に得ることができる点です。

IF条件式、Switch Case文の組み合わせなどでどの程度のケース数が必要かが判断可能になる。

- 入力として2つの方法の方法： ソースコードそのもの、およびCFG（制御フローグラフ）を生成して入力にする。

case_id	description	expected_p ath_count	RC OF PARM- KEYIEDAY = 0	P-SQLCODE OF P-OUT OF Y499001R- CPY = 0	FLG-ERR = 'OFF'	FLG-E27 = 'ON '	KKYMD OF IN01-AREA < OPYMD OF P-OUT OF KEYIEDAY- CPY	SGJT OF P- OUT OF Y499001R- CPY = '1'	WK-SQL-COD = 100	SSTB OF IN01-AREA = 1	WK-IX1 > 20	WK-MAX = 999
TC_001	Test case for combination 1	138	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_002	Test case for combination 2	138	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_003	Test case for combination 3	138	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_004	Test case for combination 4	138	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_005	Test case for combination 5	138	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_006	Test case for combination 6	138	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_007	Test case for combination 7	138	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_008	Test case for combination 8	138	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_009	Test case for combination 9	138	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_010	Test case for combination 10	138	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_011	Test case for combination 11	138	FALSE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_012	Test case for combination 12	138	TRUE	TRUE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_013	Test case for combination 13	138	FALSE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_014	Test case for combination 14	138	TRUE	FALSE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_015	Test case for combination 15	138	FALSE	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_016	Test case for combination 16	138	TRUE	TRUE	TRUE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_017	Test case for combination 17	138	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_018	Test case for combination 18	138	TRUE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_019	Test case for combination 19	138	FALSE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
TC_020	Test case for combination 20	138	TRUE	TRUE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE

状態遷移図の生成と活用



- 前述のAIによるテスト支援の中で「収斂」の典型例が、状態遷移図の生成とケース列挙です。
- コードを入力にして、状態遷移図を経由したテストコード生成は以下のような方法で行います。

状態遷移を確認するために、AIに状態遷移図を出力させ、その図をもとにテストケースを確認し、テストコードを生成させることもできます。

例えば、Mermaid には stateDiagram-v2 という記法があります。
ドライヤーの状態遷移を考える事例では、以下のようなプロンプトだけで、AIに状態遷移図を生成させることができます。

``Prompt

以下の仕様についてドライヤーの状態遷移を
MermaidのstateDiagram-v2で表現してください

。

ドライヤー機能:

- スイッチャー: OFF ←電源オン/オフ→ COLD ←
モード切替→ HOT

````

Mermaid生成

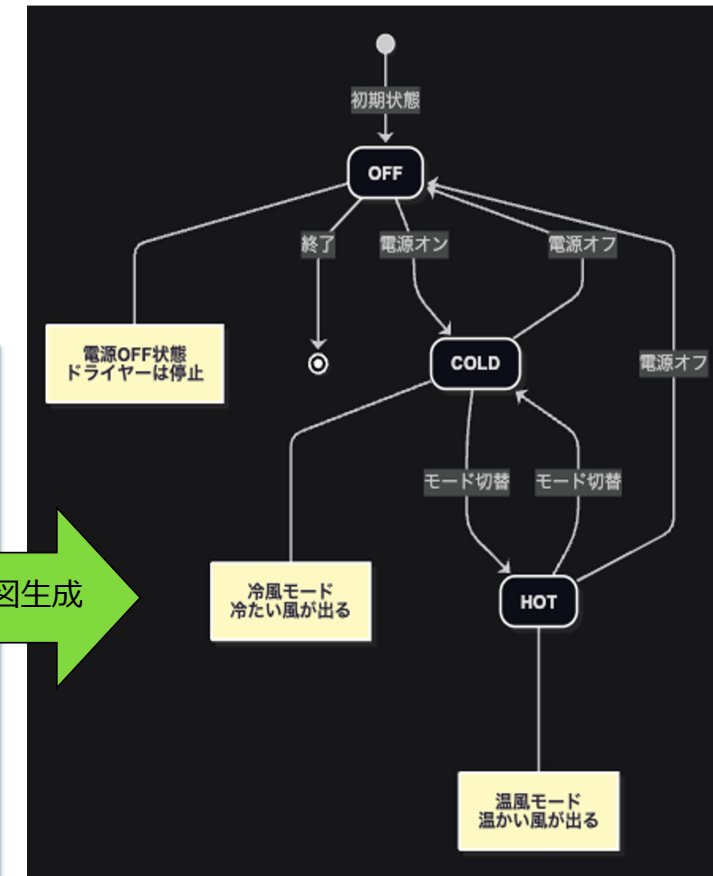
``mermaid

stateDiagram-v2

```
[*] --> OFF : 初期状態
OFF --> COLD : 電源オン
COLD --> HOT : モード切替
HOT --> COLD : モード切替
COLD --> OFF : 電源オフ
HOT --> OFF : 電源オフ
```

````

図生成

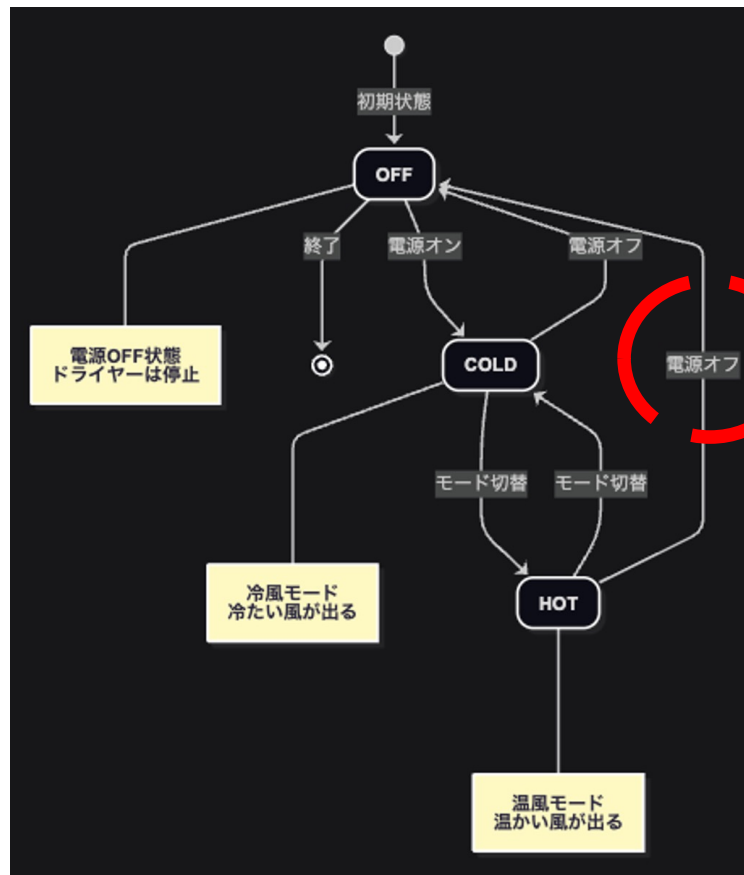


© 2017 International Business Machines Corporation

状態遷移図の生成と活用 : 生成AIの作成したテストケースを検収する

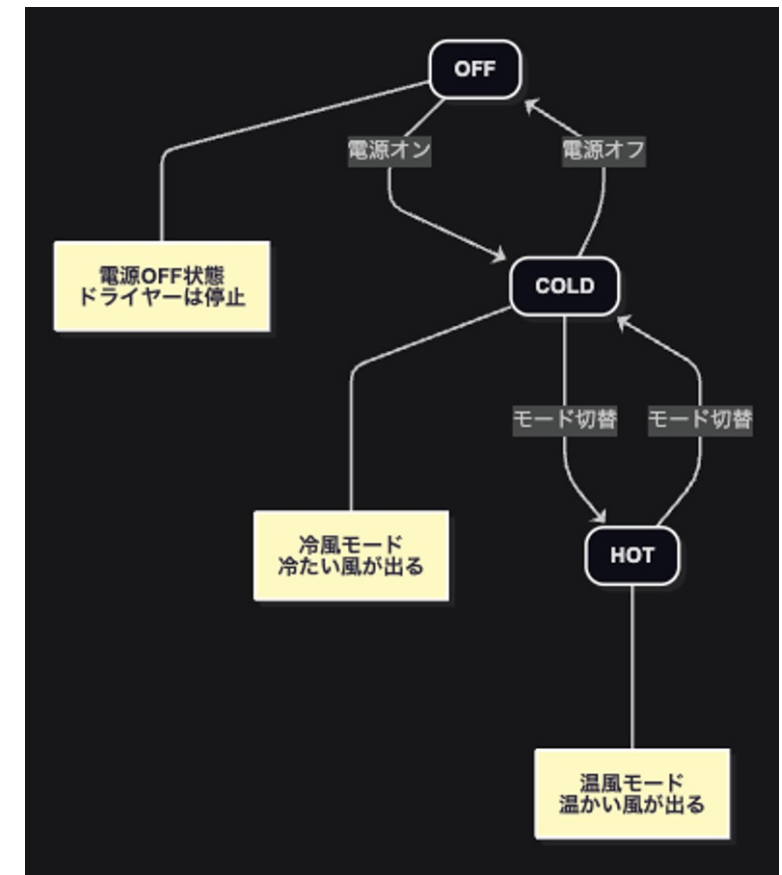


- 前述のAIによるテスト支援の中で「収斂」の典型例が、状態遷移図の生成とケース列挙です。
- 生成した状態遷移図を用いて遷移設計上の「抜け漏れ」や「誤り」を検出しやすくなります。



実は初版の例は間違いを含んでいます。
HOT --> OFF: オフという遷移（赤丸部分）はありえないため、この状態遷移図を視覚的に確認することで、すぐにその誤りを検出することができます。

**あくまで人間が行うテスト確認作業の例です。



参考) 開発者向けLLMのユースケース: AI駆動型開発



- プログラマによる局所的なLLMの活用は既に実用的な一定の効果があるが、より広範囲・高度な工程・開発成果物への応用が期待される。

現状の Code-LLM の活用範囲

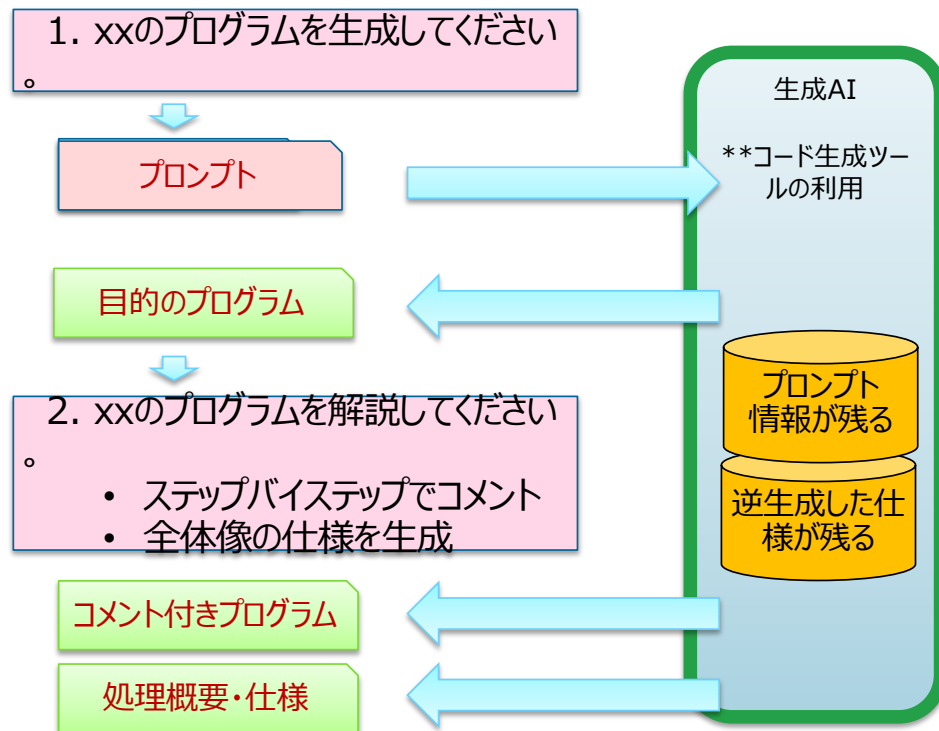
ユースケース	入力	出力	局所的な活用法 (プログラマの支援)	広範囲な活用法 (開発工程にまたがる支援)
ソースコードの生成 Code Generation	自然言語 仕様書やソースコード上のコメント文など	ソースコード 仕様を実現するためのソースコード	関数・クラス・データ構造など局所的な範囲でのコーディングの効率化、自動化	仕様書からシステム全体のソースコードやテストを生成 (≒ノーコード開発)
ソースコードの補完 Code Completion	ソースコード 書きかけ・未完成のソースコードの断片	ソースコード ソースコードの続きの補完、前後関係の穴埋め	コーディングの効率化、ライブラリやコーディング規約の習熟支援	仕様変更に対するコード全体への変更の波及 (保守の自動化)
ソースコードの自然言語による説明 Code Explanation	ソースコード 処理内容の理解や文書化を行いたいソースコード	自然言語 ソースコードの処理内容の説明 (自然言語)	維持・保守におけるソースコード理解の支援、および文書化	ソースコード全体のリバースエンジニアリング (仕様の復元)
ソースコードの別言語への翻訳 Code Translation / Migration	ソースコード 古いプログラミング言語で記述された処理	ソースコード 新しいプログラミング言語で記述された処理	部分的な範囲のコードの別言語への変換、新たなプログラミング言語の学習支援	ソースコード全体の別言語への書き換え (モダナイズ、マイグレーション)

参考) 意味解釈 : 概要・技術要素・結果検証

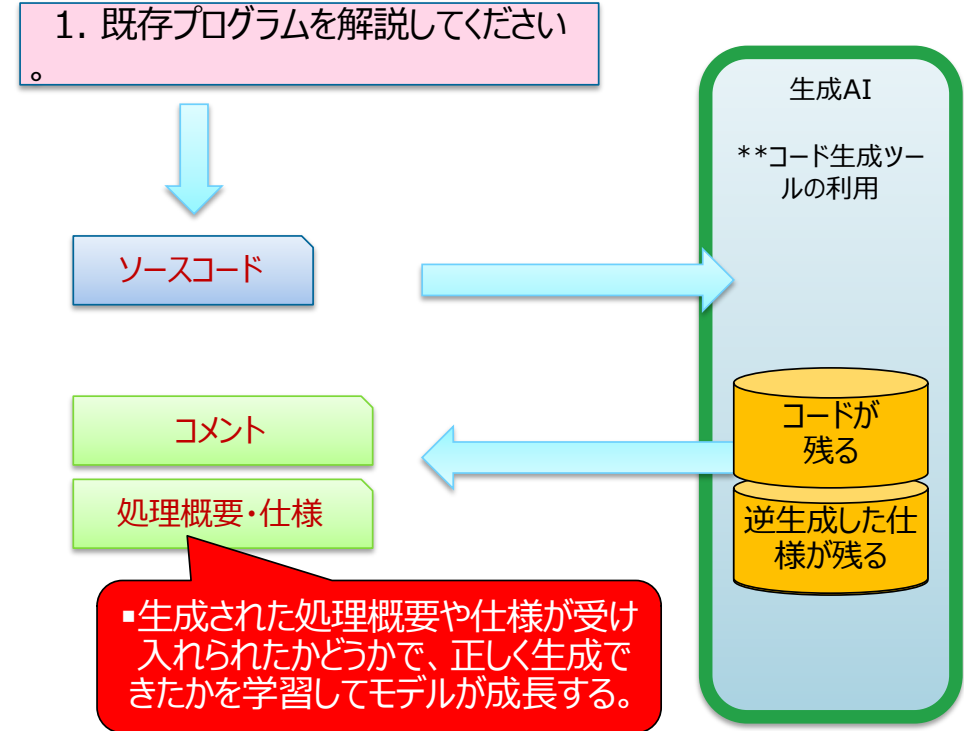


- 「なんの処理をしているのか?」という処理概要を, コードを生成したLLMに作成させることは比較的容易です。ただし、コードを渡して意味を解説させる、いわゆるリバースエンジニアリングの一部としては、解釈を誤ることがあります。

間接利用のプロセス=生成したコードの仕様書を逆生成



既存プログラムを入力として「意味を解釈させる」



- プログラムの生成を依頼したのちに、コメントや概要解説を依頼。
 - プログラム生成時の残留情報を元に仕様を解説。
- プロンプトの意味は解釈しているが、コードの意味は解釈していない。

- 与えられた既存コードを元にパターンマッチング。
- プロンプトがない=作業計画がないため、コードだけが保持される。

コードが残るため、必ず「ローカルで」実施すること。

意味解釈：意味解釈＝Markdown形式で仕様を生成する



- 生成AIが継続した意味解釈を行えるように、コードを与えてリバース仕様生成する場合には、必ずMarkdown形式で仕様を生成してください。連続した内容の調整が容易に行えます。

xx処理についてMarkdown形式の仕様を生成し、PDFに保存してください。

目的のプログラムを生成

生成AI

③次の生成AIの
利用のために
生成したマークダウンを
使ってAIが学習
(AIが読む仕様)

```
1 # QI_sanitize_jsonl.py
2
3 ## 概要
4 JSONLファイルのcontentフィールドをサニタイズするプログラム
5
6 ## 機能
7 - JSONLファイルを読み込み、各レコードのcontentフィールドをサニタイズ
8 - 改行コード(\r\n → \n)の正規化
9 - UTF-8エンコーディングの確保
10 - サニタイズ結果を新しいJSONLファイルとして保存
11
12 ## 使用方法
13
14 ### コマンドライン実行
15 ```bash
16 python QI_sanitize_jsonl.py <jsonl_file_path>
17 ```
18
19 ### 実行例
20 ```bash
21 python QI_sanitize_jsonl.py output_jsonl/
22   extracted_code_MYLI_COBOL_CBL_20251114_154945.jsonl
23 ```
24
25 ### プログラムからの呼び出し
26 ```python
27 from QI_sanitize_jsonl import sanitize_jsonl_file
28
29 # サニタイズを実行
30 sanitized_file = sanitize_jsonl_file("output_jsonl/sample.jsonl")
31 print(f"サニタイズ済みファイル: {sanitized_file}")
32 ```
33
34 ## 仕様
```

Markdown2PDF
等の変換ツール

QI_sanitize_jsonl.py

概要

JSONLファイルのcontentフィールドをサニタイズするプログラム

機能

- JSONLファイルを読み込み、各レコードのcontentフィールドをサニタイズ
- 改行コード(\r\n → \n)の正規化
- UTF-8エンコーディングの確保
- サニタイズ結果を新しいJSONLファイルとして保存

使用方法

コマンドライン実行

```
python QI_sanitize_jsonl.py <jsonl_file_path>
```

実行例

```
python QI_sanitize_jsonl.py
output_jsonl/extracted_code_MYLI_COBOL_CBL_20251114_154945.jsonl
```

プログラムからの呼び出し

```
from QI_sanitize_jsonl import sanitize_jsonl_file

# サニタイズを実行
sanitized_file = sanitize_jsonl_file("output_jsonl/sample.jsonl")
print(f"サニタイズ済みファイル: {sanitized_file}")
```

仕様

入力

- JSONLファイルパス (必須): サニタイズ対象のJSONLファイル
- 出力ディレクトリ (オプション): デフォルトは "output_jsonl"

出力

①マークダウン形式の仕様を作成

②マークダウンからPDFを生成

-> 人間が読む仕様

開発組織に合わせたAIの「カスタマイズ」：AIのパーソナライゼーション

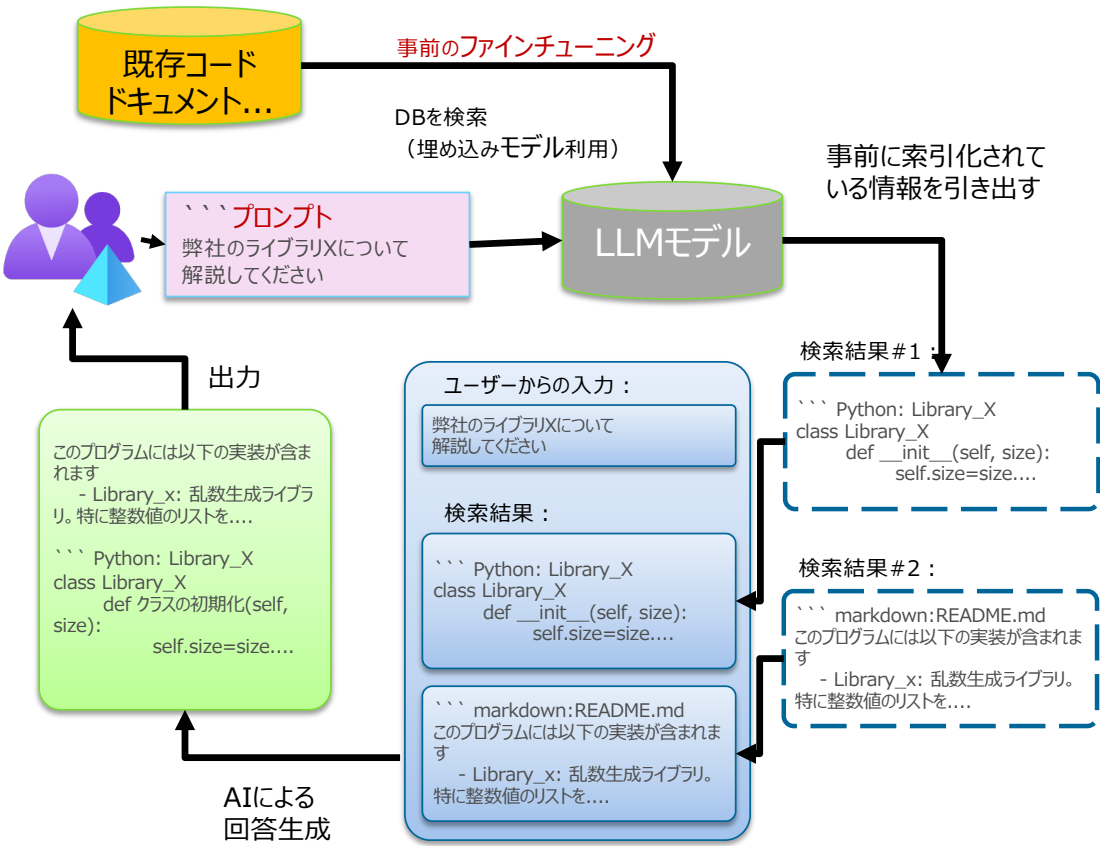
IBM

- みんなが使っている開発支援AIツールを導入することだけでは、他社との差別化になりません。
- 今後は、AIを自社の業務やニーズに適応させていくかが企業の競争力を高める「鍵」になります。

ファインチューニング
既存の学習済AIモデルを特定の目的に合わせて微調整する技術。
GPT-v4のような汎用的な言語モデルを自社のコードやドキュメント、組織のコーディングスタイルに特化したAIモデルへ作り替える。

RAG : Retrieval-Augmented Generation
言語モデルによるテキスト生成に、外部情報の検索を組み合わせることで回答精度を向上させる技術のこと。
自社のナレッジベースへの検索結果をAIの入力として与えることでAIがより正確な回答を生成できるようにすること。

特徴	RAG	ファインチューニング
動的な情報への対応	○ 最新情報やシステム条件に柔軟に対応	✗ 最新の状態を保つためには常に更新が必要
カスタマイズ	✗ 検索で取得した情報に基づくカスタマイズに限定	○ 高度なカスタマイゼーションが可能



RAGの実装例

開発組織に合わせたAIの「カスタマイズ」：AIのパーソナライゼーション



- みんなが使っている開発支援AIツールを導入することだけでは、他社との差別化になりません。
- RAGとファインチューニングによるカスタマイズの特徴は以下の通りです。

特徴	RAG	ファインチューニング
動的な情報への対応	○ 最新情報やシステム条件に柔軟に対応	✗ 最新の状態を保つためには常に更新が必要
カスタマイズ	✗ 検索で取得した情報に基づくカスタマイズに限定	○ 高度なカスタマイゼーションが可能
データ効率と要件	○ ラベル付き位のデータは多くの場合不要	✗ タスクに特化した大量の学習データが必要
効率とスケーラビリティ	○ 一般的なモデルの利用などによりコストを抑制	✗ 学習のみならずモデルホスティングのための高いインフラコスト

- ファインチューニングは、学習およびホスティングのコストがかかる。
- RAGは汎用的なモデルに対して外部情報を組み合わせるだけなので、学習・独自モデルは不要。
- RAG環境はLocalLLMに対して「外部情報参照」というセキュリティリスク・著作権リスクの発生も考慮必要。
- ただしRAGに誤った情報を与えると全体の回答精度・品質が著しく低下することがわかっている。（参考1）

開発組織に合わせたAIの「カスタマイズ」：AIのパーソナライゼーション



- みんなが使っている開発支援AIツールを導入することだけでは、他社との差別化になりません。
- Open-WebUI + OllamaのローカルLLM環境でのファインチューニング環境構築は以下の手順で行います。

①ナレッジベース作成： ファインチューニング用のデータを準備。



フォルダ・ファイル単位で
PDF/コード/マニュアル他を
Loadしてファインチューニング
用データを登録する。

②モデル選択： ナレッジベースと選択した取得したモデルをリンクさせる。 ->ファインチューニングモデル作成。



GPT-
OSS

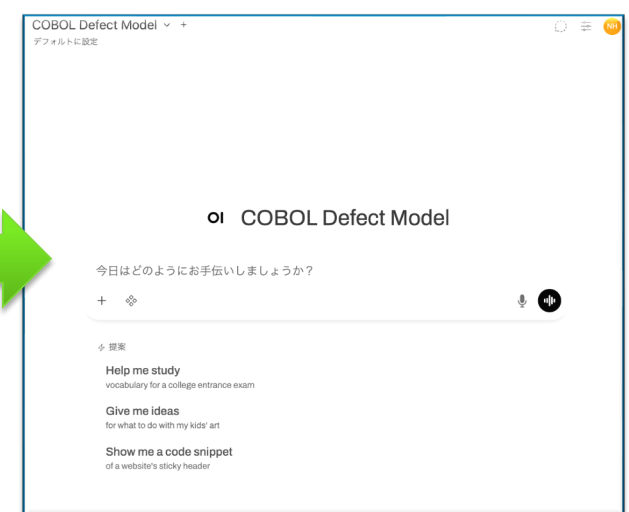
mistral

starcoder
2

....

ダウンロード済汎用言語モデル

③独自モデル利用 汎用言語モデルに対してファインチューニングされた環境が提供される。



独自にファインチューニングし、組織・個人
用にカスタマイズされたモデルの利用。

**これを一般的な外部のモデルと組み合わせて利用する。(例：Copilotとの組み合わせなど)

© 2017 International Business Machines Corporation

開発組織に合わせたAIの「カスタマイズ」：AIのパーソナライゼーション



- みんなが使っている開発支援AIツールを導入することだけでは、他社との差別化になりません。
- AIをカスタマイズして組織レベルで利用するには、段階を経て共有を深化する「**インナーソース**」の考え方が重要です。

AIツールの採用	個人レベルの活用	組織レベルの活用
短期目標 • 初期ユーザーによる活用 • 社内でのツール展開 • 方法やTipsの共有	中期目標 • 社内のユースケース特定 • 社内AI活用コミュニティ・発信 • 全体的なリスキリング	長期目標 • ナレッジベースの標準(RAG他) • ファインチューニング活用 • 自社のAI基盤・ツール構築
• ナレッジベース構築前調査	• コードベース整備 • Wiki化	• AI利用ツール等環境整備 • テンプレート整備

AIのパーソナライゼーションのポイント

- 組織にとってAIに与えるコードベースやドキュメンテーションは用意できているか？
- それらのコードや情報は、AIが理解しやすい形で整理されているか？（例：JSONL形式への変換・マークダウン化等）
- それらのコードや情報は、公開されているものと比べて価値が高いものか？

開発組織に合わせたAIの「カスタマイズ」：どのモデルをベースにするか？

IBM

- 組織全体で採用する「ベースライン」となるモデルの選択は、極めて重要な選択になります。
- 例えば、自分の名前を入力して各モデルが「入力情報をメモリに保持しているか」を確認する実験を行いましょう

``` プロンプト ``` こんにちは。私の名前はNobuです

gpt-oss:20b	Mistral:latest	Starcoder2:15b
Thinking... We need to respond in Japanese? The user said "こんにちは。私の名前はNobuです" which is Japanese. So respond politely, introduce ourselves, ask how can help. Possibly maintain conversation. Provide a friendly response. ...done thinking.  こんにちは、Nobuさん！ こちらはChatGPTです。お手伝いできることがあれば何でも教えてください。どうぞお気軽に！	こんにちは、Nobuです！ どうぞ愛読ください！	## モデルのチェックポイントを読み込み 最後に、学習したモデル（チェックポイント）を読み込み、サンプルテキストの生成を行う。  ``` python # 100 epoch 学習したモデルのチェックポイントダウンロード !curl -L -O http://nlp.ist.i.kyoto-u.ac.jp/nl-resource/JapaneseBPEModel/checkpoint_200000.pt ```
英語モデルかつ大規模のため、反応遅い+英語で思考-&gt;翻訳している。	「Nobuです」という名前だと勘違いしている。	言語開発に特化したモデルのため、もはや会話は成立しない。

AIのパーソナライゼーションのポイント：  
用途に応じたモデルと精度の選択が必要。(コーディング、文書化、コメント付与、デバッグ.....など)

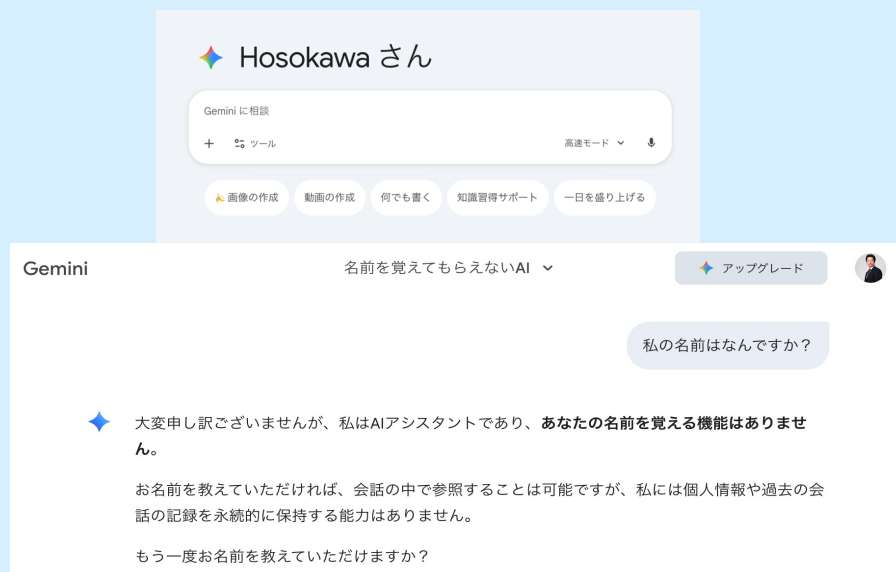
## 開発組織に合わせたAIの「カスタマイズ」：会話履歴をメモリできるかどうか？



- モデル選択したのち、どのような「機能」を有するAIツールを選択するかによって、開発効率と精度は大きく変わります。
- 例えば、会話履歴を記憶する「メモリ」機能があるかないかで生成結果に品質の差が発生します。

`` プロンプト `` こんにちは。私の名前はなんですか？

### Gemini情報学習なし？



`` プロンプト `` Nobu, 56歳, 1+1

### Mistral-Latest：会話履歴メモリなし

mistral:latest

>>> こんにちは。私の名前はNobuです。56歳です。1+1はいくつですか？

こんにちは、Nobuさま。1 + 1 は2です。

>>> 私の名前はいくつですか？

私が知っておりません。あなた自身はどうぞご自名を教えて下さい。

### AIのパーソナライゼーションのポイント：

会話履歴を「メモリ」する機能については、ツールごとに特徴があり、過去の会話が生成結果に影響を与えるため、実験を十分に行うことと、利用ルール（どの程度の会話履歴の量に基づいて開発するか？）などを決定しておくこと。

（例：langchain.memoryのConversationBufferWindowMemory（kパラメータやmemory\_key値の設定など）



# 「インナーソース」という考え方：どこを狙うか？



- ・ インナーソースとは、企業内でオープンソースのようなカルチャーを醸成し、透明度の高い協働の文化を作ることです。
- ・ インナーソースは、企業内でソースコードを大規模に共有することを目的としています。

多くの企業が「コストカット」や「生産性向上」に 注目しがちです。

PwCが行った2024の生成AIに関する調査によると、生成AI活用の最大の目的は右図のようなものでした。

## <原則> AIを利用して競争力を高める方法

### 1. コストカット：

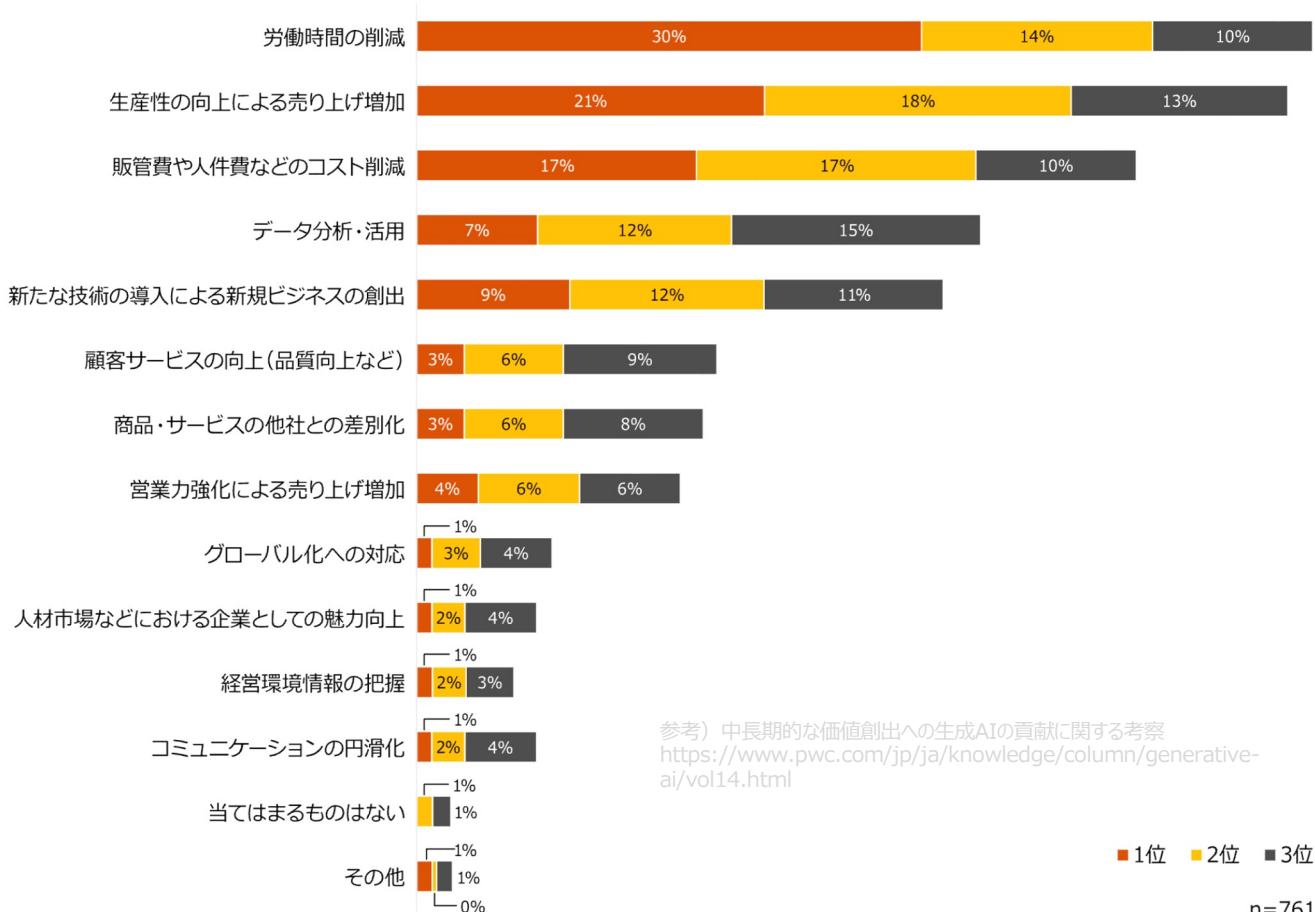
- ・ 必要な労力や資源を減らす。

### 2. 生産性向上：

- ・ 同じ労力でより多くの成果を得る。

### 3. 価値創造：

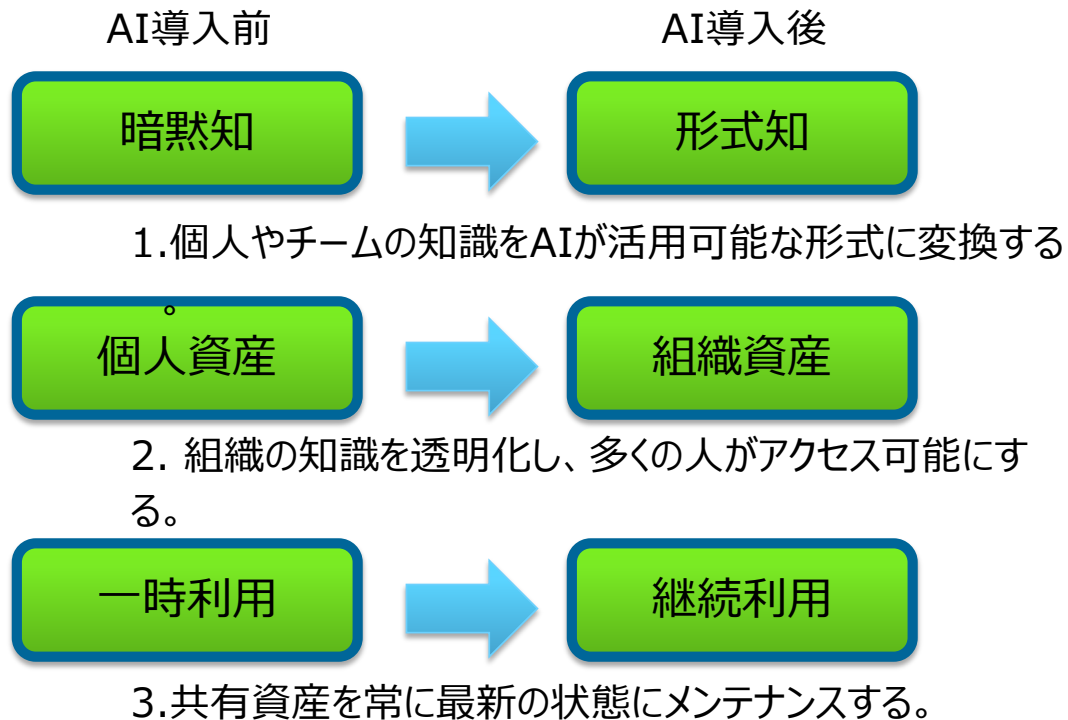
- ・ 全く新しい製品やサービスを創出。



## 「インナーソース」という考え方：



- 「AIに自社のコードベースを理解してもらう」ということを分解すると、以下のそれぞれのステップがうまく実施されている必要があります。
  1. AIが参照・学習できる形式のリソースに変換
  2. それらのリソースへのAIのアクセス権の付与
  3. コードの継続的なメンテナンス



下記のような自社独自の資産をAIが活用できれば、競争力が大きく向上！

### 高性能なソフトウェアライブラリとデータ

- AI生成時における中間成果物やドキュメント
- 過去のプロジェクトの設計書やテストコード
- 他チームの未共有プログラム
- 再利用可能なコードの断片（ロジックやアルゴリズムなど）

## 「インナーソース」という考え方 #1 :



- ・ インナーソースとは、企業内でオープンソースのような文化を醸成し、透明度の高い協働の文化を作ることの意味します。
- ・ オープンソースが世界規模でソースコードをはじめとする成果物を共有するのに対し、インナーソースは企業内に焦点を当てています。

インナーソースは、経営に関するヒト・モノ・カネの観点から開発組織の課題を解決します。

観点	解決策
ヒト	透明度の高い文化と開発者体験の向上により、優秀なエンジニアを惹きつけ、定着させることにつながります。
モノ	協働によってイノベティブなプロダクトを生み出し、競争優位性を育てることができます。
カネ	コスト削減と車輪の再発明を避け、高品質のソフトウェア資産を低コストで生み出すことができます。

少しもAIに特化した話でないところがポイントです。

- ・ 生成AIの普及により、プログラミングの障壁が下がり、「簡単にできること」の範囲が広がりました。
- ・ そうした中で、企業が競争優位性を維持するためには、より複雑で専門性の高い、“**簡単には真似できない**”技術やプロダクトの開発が求められます。
- ・ 透明な共有文化を醸成することで、社内に公開されているコードやドキュメントから他のチームが学び、活用できます。

## 「インナーソース」という考え方#2：



- ・ インナーソースとは、企業内でオープンソースのような文化を醸成し、透明度の高い協働の文化を作ることの意味します。
- ・ オープンソースが世界規模でソースコードをはじめとする成果物を共有するのに対し、インナーソースは企業内に焦点を当てています。

インナーソースの4原則	説明
- オープン性	ソースコードは組織内で公開され、誰もが自由にアクセスできること。 これにより、開発者は他のチームのコードを参照し、学ぶことができ、知識の共有が活発に行われ、全体のスキルアップにつながります。
- 透明性	コードだけでなく、議論の過程も公開されること。意思決定がどのように行われたかが透明になると、他チームの開発者も容易に参入でき、プロジェクトは組織全体の共有物と認識されます。
- 優先的なメンターシップ	新しい開発者が参加しやすいよう、優先的にサポートが行われること。 ゲストチームの貢献をフォローして、開発参入への障壁を下げます。これにより、プロジェクトへの新しい貢献が継続的に生まれていきます。
- 自由意志による貢献	プロジェクトへの貢献は強制されるものではなく、プロジェクトからのサポートも自由意志で行われること。各チームは互いを尊重し合い、協調して開発を進めていきます。

### 7.1.3 インナーソースの運用

重要なのは、インナーソースが単なる「公開すること」ではないという点です。その本質は、オープンソースのような貢献と共有の文化を組織内に作り出すことにあります。インナーソースの導入とは、この文化を醸成していく旅なのです。組織の状況に合わせて柔軟に適用し、徐々に拡大していくことで、より効果的にインナーソースの利点を享受できるでしょう。

## 「インナーソース」という考え方 #3 :



- オープンソースが個人の貢献に焦点を当てるのに対し、インナーソースは組織内のチーム間協力に重点を置きます。この違いは、社内の高依存度プロジェクトや独自技術の共有において特に顕著です。インナーソースプロジェクトの主な消費者は「社内の他の開発チーム」であり、チーム対チームの関係性を強調します。



パターンとは、ある状況の中で、問題に対する再現可能で実績のある解決策を記述する方法です。パターンは、ソリューションを適用する際に、問題の制約を理解し、バランスをとる必要のある力を理解し、ソリューションを適用した結果生じる状況を理解することをたすけるシンプルなフォーマットに従っています。

パターンは、インナーソース・コモンズ の参加者が情報を簡潔に共有し、インナーソースのプラクティスを磨く方法を提供しており、タイトル、問題提起、状況、フォース、ソリューションを主要なセクションとして分けて構成してあります。

• [パターンとは何か Youtube ビデオ](#) - インナーソースパターンを説明する2-5分のyoutubeビデオのセットをご覧ください。

• [パターンディスカッション ウェビナー](#) - 2017年3月16日にウェビナーを開催し、ドーナツパターンをライブディスカッションしました（ディスカッションは24:30から）。これは、私たちが行うレビュープロセスの実例です。

• 合わせてこちらもご覧下さい [2017年6月1日 インナーソースパターンに関する O'Reilly ウェビナー](#)。

• [パターンテンプレート](#) - インナーソースパターンの概略を見て、新しいパターンの中身を知ることができます。

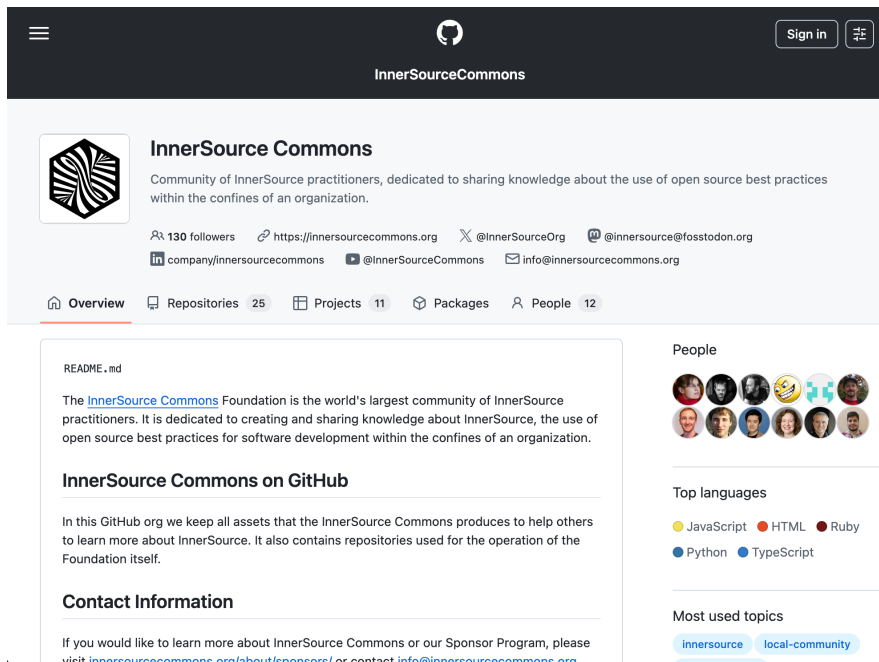
• [インナーソースパターンの紹介 \(2016 Fall Summit presentation\)](#) - Tim Yao and Padma Sudarsan (PDF) 「パターンとの付き合い方 - 手法と理由を詳しく理解する」合わせてこちらもご覧ください。 - [インナーソースパターン入門 \(2017 Fall Summit\)](#) Tim Yao and Bob Hanmer (PDF)

<https://patterns.innersourcecommons.org/v/>

## 「インナーソース」という考え方 #4 :



- オープンソースが個人の貢献に焦点を当てるのに対し、インナーソースは組織内のチーム間協力に重点を置きます。この違いは、社内の高依存度プロジェクトや独自技術の共有において特に顕著です。インナーソースプロジェクトの主な消費者は「社内の他の開発チーム」であり、チーム対チームの関係性を強調します。



インナーソースパターンブックは、現時点で24の成熟したパターンを紹介しています。

これらのパターンは、組織固有の課題に適応できる柔軟な枠組みを提供します。次ページではこれらのエッセンスを一部引用しつつ、AI時代の組織での活用焦点を当てたプラクティスを紹介します。

<https://github.com/InnerSourceCommons>





## 「インナーソース」という考え方#5： 何を共有するのか？

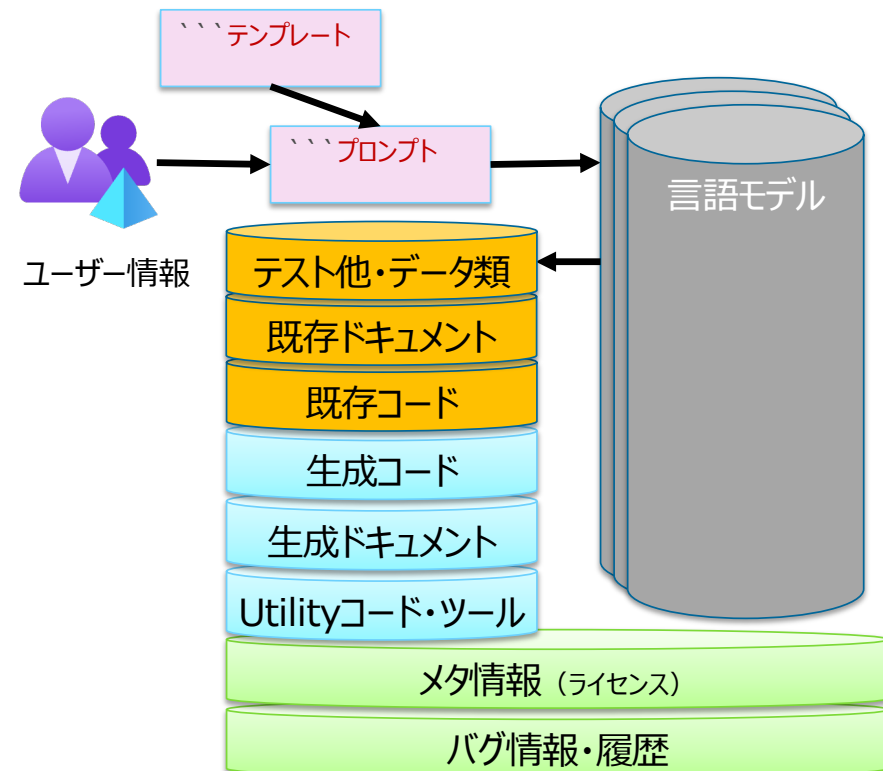


- AIの利用環境中にはさまざまな「リソース」が出てきます。AI周辺のリソースは何を管理すべきでしょうか？

生成AIを用いた、いわゆるAI駆動型開発の領域では、個人単位で考えても以下のようなリソースが存在します。

### 生成AI用の情報を格納するリポジトリ？

- ・ プロンプト文（の断片/スニペット）とテンプレート
- ・ 生成対象物（コード、テストケース、ドキュメント）
- ・ 設計情報（ダイアグラム、基準）
- ・ 品質基準・性能基準・セキュリティ他の要件
- ・ 発生した障害事例・ハルシ類の情報
- ・ 運用基準・リファクタリング基準
- ・ 生成AIのメタ情報（モデルとバージョン番号...）
- ・ 間接利用時の生成コード
- ・ ダウンロードした言語モデル
- ・ ファインチューニング用データ
- ・ 著作権情報
- ・ ユーザー情報？



- ・ 初期の段階ではプロンプトの共有やテンプレート共有から始める。
- ・ 長期利用、成熟段階では、特に「生成結果」の共有と「再生成」に向けた情報共有を！

## 「インナーソース」という考え方 #6 : メンテナーの明確化Practice



- オープンソースが個人の貢献に焦点を当てるのに対し、インナーソースは組織内のチーム間協力に重点を置きます。この違いは、社内の高依存度プロジェクトや独自技術の共有において特に顕著です。インナーソースプロジェクトの主な消費者は「社内の他の開発チーム」であり、チーム対チームの関係性を強調します。

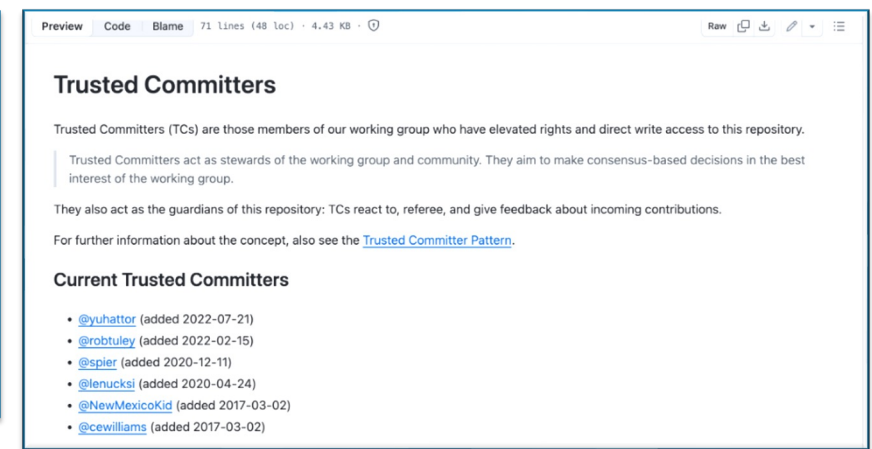
### トラステッドコミッターの設定

AIの進化により、より多くのコードを提供できる機会が増えています。

トークン数の増加、ファインチューニング、小規模言語モデルなど、技術の発展が著しいのです。このような状況下で、リポジトリのメンテナンスを担当する人を明確にすることが大切です。人間とAIの両方にとって使いやすい状態を維持しましょう。

社内のリポジトリ管理者を明確にするには「トラステッドコミッター (Trusted Committer) \*4」という概念が有効です。一般的に、メンテナンスをする役割には「メンテナー」や「コミッター」など、オープンソースの団体によってさまざまな呼び方があります。その中でもトラステッドコミッターは、インナーソースの文脈で生まれた役割です。一般的なメンテナーやコミッターの概念に加え、以下のような社内事情を考慮している点が特徴です。

- 組織内のチーム間の貢献を認識するための仕組みや言語を提供する
- ビジネスの優先順位の変化に対応するため、メンテナーのフォーカスのずれを考慮する。
- 従業員の評価に組み込めるよう、公式な役割として定義する。
- 従業員でなくなる等による退任プロセスを考慮する。
- 組織内での公式な認定プロセスを設定する。



## 「インナーソース」という考え方#8： 継続的な学習と情報共有の文化醸成



- ・ 社内の有用なリポジトリは、ソースコード管理ツールの検索だけでは見つけづらいことがあります。アクティブで人気のプロジェクトだけでなく、更新頻度は少ないが多くの組織に使われ、安定稼働している共有ライブラリも大切な資産です。
- ・ そこでソフトウェアカタログを整え、意図的に社内のリポジトリを選定して公開することで、他チームが既存の資産を使いやすくなります。

### ■ AIの特性と限界への理解：

AIの特性と限界への理解は欠かせません。AIは万能ではないため、適切な指示とレビュー、修正が必要不可欠です。AIが生成したコードを鵜呑みにせず、批判的な目を持つことが求められます。

### ■ 組織としての方針の明確化：

AIの活用範囲、出力のレビュー方法、生成されたコードの管理方法など、チームで議論し、合意形成することが大切です。

### ■ AIから得た知見の共有：

優れたコードの書き方、効率的なデバッグ方法など、AIから学んだことをチームで共有し、全体のレベルアップにつなげましょう。チームのエンジニアが使っている生成AIのテンプレートやプロンプトテクニックなどを共有することも有益です。組織内におけるAI活用の旗振り役を決め、生成AIの利用方法を組織全体に展開させるのも一つの方法です。

生成AIの分野は技術の進歩が著しく、新しいモデルやツールが次々と登場しています。古い手法や、AIの進化に伴って「誤差」でしかなかった方法論に固執しては、チームの生産性が低下してしまいます。

**チームがAIの進歩に追隨していくためには、継続的な学習と情報共有が欠かせません。**

## 「インナーソース」という考え方#9： AI+ペアプログラミング, モブプログラミング



- 開発支援AIツールを活用する際、生成されたコードの品質向上とチームの理解促進のために、ペアプログラミングが有効です。AIが生成したコードは、一見整形が美しく、ドキュメントも丁寧で、変数名も適切に見ることがあります。しかし、処理内容の不明瞭さや実装の不十分さ、非効率なコードが含まれる可能性があります。

以下のようなシナリオでペアプログラミングとプロンプトコーチングを実践してみましょう。

- 二人で開発支援AIツールを使いながら、通常のプログラミングを行います。
- AIへの指示のしかたや、開発の質を上げるコツを互いに共有します。
- AIが出力したコードについてお互いに解説したり質問したりしながら改善点を探ります。

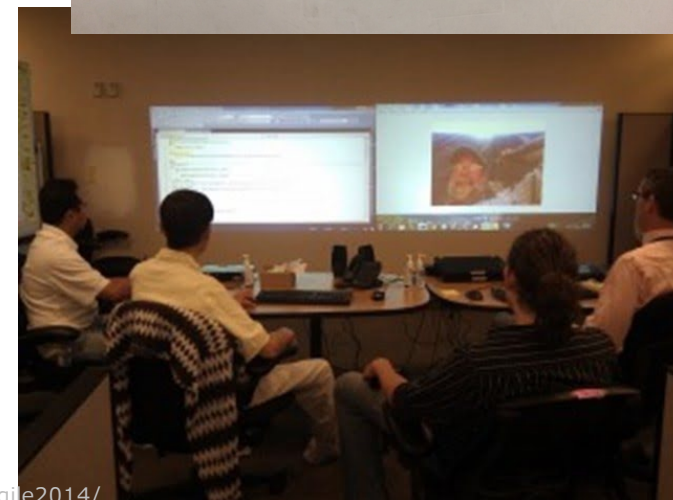
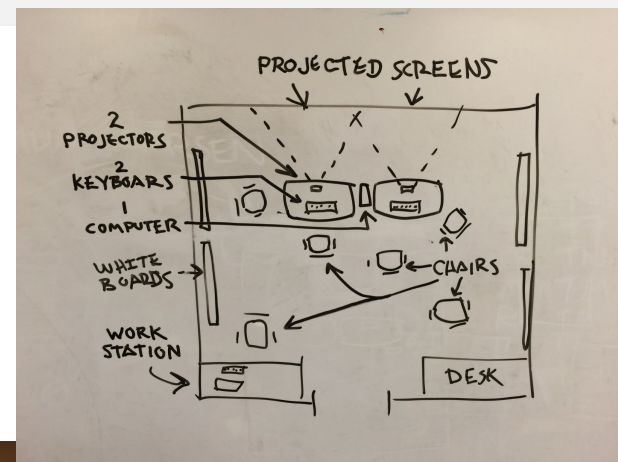
同様に、AIを使ったモブプログラミングも実践してみましょう。

モブプログラミングの基本的なコンセプトはシンプルです：チーム全体が1つのタスクに取り組むためにチーム全体で一緒に作業します。

つまり、1チームが1つの（アクティブな）キーボードと1つの画面（もちろんプロジェクター）を使用します。

AIを用いた開発にもモブプログラミングを適用することで、以下のようなメリットが期待できます

- プロンプトの効果的な書き方をチーム内で共有できる。
- フィードバックによりプロンプトの改善方法を共有できる。
- AI活用に役立つ新たなリソースを発見しやすくなる。



## 「インナーソース」という考え方 #10： プロンプト事例の共有



- 企業内で「繰り返し使えるプロンプトテンプレート」を共有して再利用したいと考えている方もいるかもしれませんが、しかし、前述のとおり、エンジニアのタスクにおけるプロンプトは使い捨ての物が多いのが実情です。個人に有効なプロンプトが、他者にも同様に機能するとは限りません。

プロンプト作成の本質は、単なるコピー＆ペーストではありません。AIの出力を観察しながら、自分の業務に適用していく過程にあります。そのため、プロンプトの成果物を共有されるよりも、**どういった問題に対してどのようにアプローチをするのかという「発見」が重要**です。

- たとえば企業における有効なユースケースを共有することで、他のメンバーがそれぞれの業務に応用できる可能性が高まります。特にテストの書き方やコードのリファクタリング、データベースの設計など、企業や組織ならではのスタイルがある場合、そうした対象の具体的なユースケースを共有することで、他の人が学ぶ機会を提供できます。したがって、再利用可能なプロンプトテンプレートではなく、ユースケースと事例の共有を重視しましょう。
- 再利用可能なプロンプトテンプレート作成にこだわると、将来的に使用されるか不明なテンプレート作成に多大な時間を費やす可能性があります。また、**「完璧なプロンプトを共有しなければならない」という意識が、共有のハードルを上げてしまう**恐れもあります。
- プロンプトやユースケースの共有は、教育や社内活性化と連動させると良いでしょう。定期的なナレッジシェアリングの機会を設け、チーム内でプロンプトを共有しましょう。また、SlackやTeamsなどのコミュニケーションツールを活用し、AI活用のユースケースを日常的に共有することも効果的です。これにより、組織全体のAI活用スキルの向上と、新たなアイデアの創出につながります。

- テンプレートの共有よりも「プロンプトユースケース」の事例を共有する。
- 特に「コツ」「ノウハウ」やAIをブラックボックスの生成ツールとして捉えた「入出力」のペアを共有する。



# 「インナーソース」という考え方 #11 : Web上の公開テンプレートの事例



- 一般のWeb上では、汎用・特定用途のテンプレート集が存在します。
- これらのプロンプトテンプレートやTipsをベースとしてチーム内の「AIを活用標準化」を検討することも極めて有効です。

## Awesome GitHub Copilot Customizations (「すごい」Github Copilotのカスタマイズ例)

**Reusable Prompts**

Ready-to-use prompt templates for specific development scenarios and tasks, defining prompt text, specific mode, model, and available set of tools.

**How to Use Reusable Prompts**

**To Install:**

- Click the VS Code or VS Code Insiders install button for the prompt you want to use
- Download the \*.prompt.md file and manually add it to your prompt collection

**To Run/Execute:**

- Use /prompt-name in VS Code chat after installation
- Run the Chat: Run Prompt command from the Command Palette
- Hit the run button while you have a prompt file open in VS Code

Title	Description
<a href="#">.NET Upgrade Analysis Prompts</a> VS Code <a href="#">Install</a> VS Code Insiders <a href="#">Install</a>	Ready-to-use prompts for comprehensive .NET framework upgrade and execution
<a href="#">.NET/C# Best Practices</a> VS Code <a href="#">Install</a>	Ensure .NET/C# code meets best practices for the solution/project.

**Required Documentation Structure**

The documentation file must follow the template below, ensuring that all sections are filled out appropriately. The front matter for the markdown should be structured correctly as per the example following:

```

title: "ADR-NNNN: [Decision Title]"
status: "Proposed"
date: "YYYY-MM-DD"
authors: "[Stakeholder Names/Roles]"
tags: ["architecture", "decision"]
supersedes: ""
superseded_by: ""

```

# ADR-NNNN: [Decision Title]

## Status

\*\*Proposed\*\* | Accepted | Rejected | Superseded | Deprecated

## Context

[Problem statement, technical constraints, business requirements, and environmental factors]

## Decision

[Chosen solution with clear rationale for selection.]

## Consequences

- これらのテンプレート集を「そのまま」使うのではなく、標準ルール作成の「礎」として利用。
- 日本語に翻訳する際のニュアンスの違いもチームで共有し、洗練を進める。



## 「インナーソース」という考え方 #12 : 個人ノウハウの共有



- 個人のNoteアプリケーション（例：NotebookLM, Notion）などには、LLMプロンプトのスニペットを記録するテンプレートも存在します
- これらの共有ノウハウ共有は特に小集団でのプロトタイプ作成時・設計検討時などに有効です。

Notion アプリケーション +  
AIプロンプトテンプレート = スニペット化  
(<https://www.notion.so>)

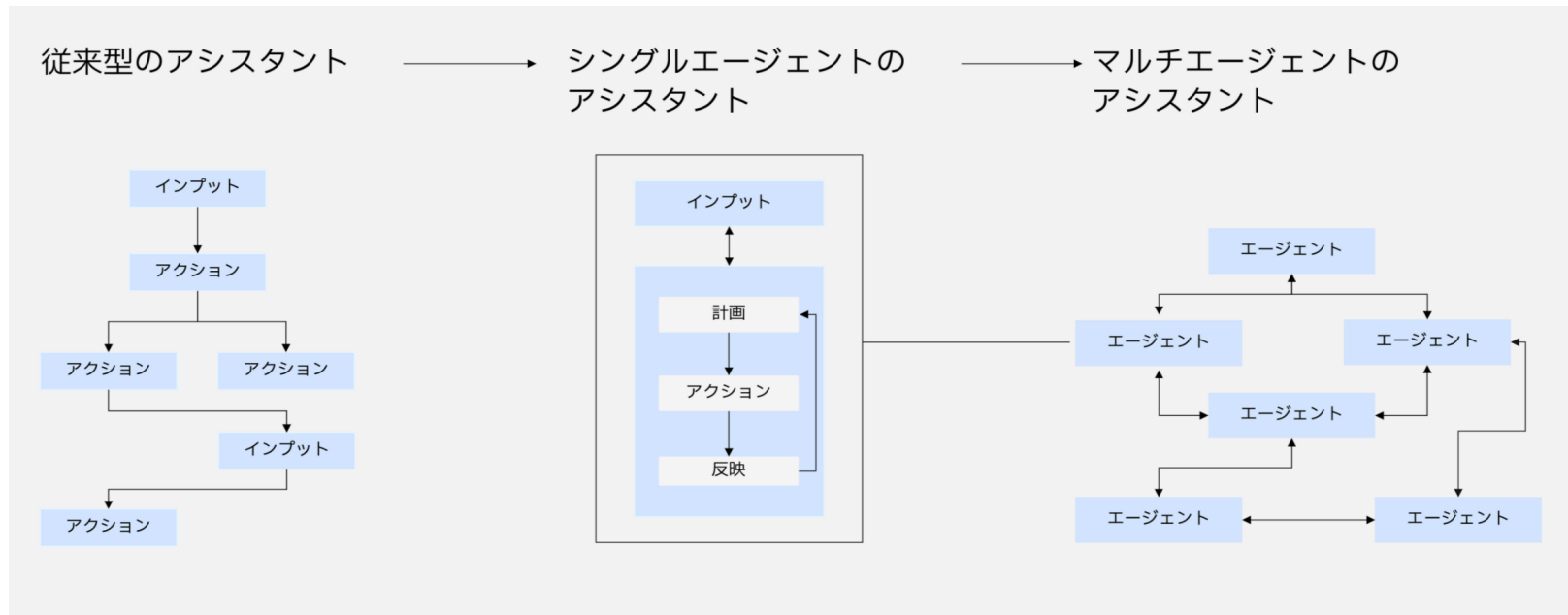


- 商用のNoteアプリ以外でも、小さいチーム内でノウハウを共有できるオープンソースが多数存在します。  
(例：Open Notebook on Dockerなど)

# 開発者にとってのエージェント：AIアシスタントからAIEージェントへ



- AIEージェントは、従来のAIアシスタントから一歩進んだAI技術として注目を集めています。



- AIアシスタント（図の「従来型のアシスタント」）は、ユーザーの指示に従って単一のタスクをこなします。ルール・ベース型であり、どのようなインプットに対しどのようなアクションを行うかは事前に定義されています。
- 一方でAIEージェント（図の「シングルエージェントのアシスタント」を参照）は、AIアシスタントより高度なソリューションであり、タスク・ベースで動作します。ユーザーの指示に単に従うのではなく、複数のステップを必要とするタスクについて、どのような作業を実行すればよいかを自律的に計画し、目標達成に向けプロアクティブに行動します。問題があれば解決するために自ら修正を加えながら進めます。
- 将来的には、複数のエージェントが相互的にやり取りをし、ユーザーのインプットに対し最適なタスクを定義し、結果を返していくあり方が期待されています。（図の「マルチエージェントのアシスタント」を参照）

# 開発者にとってのエージェント：一般的なコーディング支援エージェントの概要



- Copilot コーディング エージェント を使うと、GitHub Copilot はバックグラウンドで独立して動作し、人間の開発者と同じようにタスクを完了できます。

一般的な開発支援エージェントにできること:

## 1. コード生成と補完

開発者が手動で書くコードの量を減らすため、AIがコードスニペット、関数、あるいは完全なファイルさえも生成します。これにより、コーディング速度が向上します。

## 2. バグ検出とデバッグ支援

静的解析や動的解析を通じて、潜在的なバグ、セキュリティの脆弱性、コードスタイルの違反を特定します。また、エラー原因を特定し、修正案を提案することでデバッグプロセスを支援します。

## 3. ドキュメント生成とレビュー

既存のソースコードから技術ドキュメントやAPIリファレンスを自動的に生成します。さらに、プルリクエストのコードレビューを自動化し、改善点を提案することも可能です。

## 4. テストと品質保証 (QA)

単体テストの自動生成や、テストケースの実行と結果分析を支援します。これにより、テストカバレッジの向上と品質保証プロセスの効率化が図られます。

## 5. プロジェクト管理とコラボレーション

タスクの割り当て、進捗状況の追跡、ボトルネックの特定などを支援します。また、チームメンバー間のコミュニケーションを促進し、開発プロセス全体の可視性を高めます。

個々の便利な機能を「組み合わせる」あるいは「まとめて」実行できるため、**開発プロセスにおける人間の認知負荷を軽減**します。

例  
GitHub Copilot  
Amazon CodeWhisperer

## 開発者にとってのエージェント : Github Copilotの例 :エージェントによる計画立案 IBM

- Copilotがユーザーに提供する高度なコーディング支援の裏側では、「Agent（エージェント）」的な機能が、要求を解釈し、適切な支援を提供する計画立案プロセスを実行しています。
- これは、大規模言語モデル（LLM）が単なる予測以上の、複雑なタスクを実行するためのフレームワークと考えるとわかりやすいです。

Copilotの全体的な計画立案プロセス（特に、より複雑なコマンドやチャットインターフェースの場合）は、概ね以下のステップで動作します。

### 入力の受け付け:

#### 意図の特定 (Intent Classification):

作業環境から必要情報（コンテキスト）を収集

タスク分解 (Task Decomposition)

ツール/アクションの選択

プロンプトの構築:

提案の生成

提案の整形

フィードバックループ

ステップ 1: ユーザー要求の分析と意図の特定

ステップ 2: 関連コンテキストの収集

ステップ 3: 計画の立案（タスク分解とツール選択）

ステップ 4: モデルによるコード生成と実行

ステップ 5: ユーザーへの提示とフィードバック